AGENT-KGRAPH: Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Devanshu Jayeshbhai Aslaliya (24U020014 – Vth Semester)

Under the supervision of

Dr. Yatendra Sahu
Asst. Professor
(Department of Computer Science and Engineering)

November 2026

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	
	Declaration	
	Abstract	1
1	Introduction	2
2	Literature Review or Survey	5
3	Problem Description and Objective	13
4	Methodology & Work Description	15
5	Proposed Algorithm	24
5-A	Proposed Flowchart/ DFD/ Block Diagram	26
6	Tools & Technology Used	30
7	Result Analysis	39
8	Conclusion & Future Scope	62
9	References	64
10	PLAG REPORT (Through Turnitin) Similarity Plagiarism Report + AI Plagiarism Report	66

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"AGENT-KGRAPH: Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval"** has been satisfactorily completed by **Devanshu Jayeshbhai Aslaliya (24U020014).** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-27.

Date: 02/11/2026

Dr. Yatendra Sahu,
Asst. Professor,
Department of CSE,
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia,
Head/Department Coordinator
Department of CSE,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled "AGENT-KGRAPH: Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval" presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Yatendra Sahu**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Devanshu Jayeshbhai Aslaliya (24U020014) Sign of student

ACKNOWLEDGEMENT

I Devanshu Jayeshbhai Aslaliya, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled " AGENT-KGRAPH: Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval ".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Dr. Yatendra Sahu, for his constant guidance, invaluable suggestions, and expert advice throughout the project. His knowledge and insights helped us enhance the quality of our work, and his support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, Dr. Rahul Chaurasia, for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the Department of Computer Science and Engineering for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Devanshu Jayeshbhai Aslaliya
24U020014

AREA OF WORK

AGENT-KGRAPH combines knowledge graphs, vector retrieval, and agentic question answering. It uses graph relations to connect entities and vector search to find relevant text. The goal is to make answers easier to trace and to support questions that benefit from both retrieval methods.

The system runs as a modular pipeline. It preprocesses questions, answers, passages, and graph data, then builds the graph by extracting and verifying candidate triples and resolving entity references. Retrieval gathers graph paths and vector-based evidence. A router chooses graph, vector, or hybrid retrieval, and answer synthesis produces a response with supporting provenance. The self-evolution stage evaluates candidate facts before merging them and can decay or prune stale graph information.

The roadmap has nine batches. Batches 1–5, covering the foundation, graph construction, retrieval, agentic question answering and routing, and self-evolution, are marked complete. Batch 6 will validate the existing MetaQA import and add HotpotQA and WebQSP. Batch 7 will establish a broader evaluation protocol, Batch 8 will run systematic experiments with real models, and Batch 9 will bring together the documentation, demonstrations, and final submission.

The project has a recorded full test run with 71 passing tests. Its real-model MetaQA benchmark contains 102 questions, with 34 in each hop category. We recalculated the answer metrics from the saved predictions and references and recomputed mean latency from the per-question timings. Overall, the benchmark achieved 74.51% Exact Match, 83.54% Token F1, and a mean query latency of 7,684.04 ms. For 3-hop questions, it achieved 58.82% EM and 68.15% Token F1, with a mean latency of 19,132.57 ms. This benchmark covers a KG-routed sample; it does not compare AGENT-KGRAPH with other systems.

LIST OF FIGURES

Fig	Description	Page no.
1	Query Routing Pipeline	25
2	Knowledge Graph Construction/Update Process	25
3	Overall Hybrid Retrieval and Context Fusion Architecture	26
4	Hybrid Retrieval Architecture	26
5	Multi-Hop Question Decomposition and Answer Synthesis Architecture	27
6	Multi-Stage Data Processing, Validation, and Performance Evaluation Framework	27
7	Knowledge Graph Evolution Through Dynamic Addition, Replacement, and Removal of Triples	28
8	Evidence-First Question Answering Architecture with Evaluation and Continuous Improvement Loop	28
9	Accuracy Comparison on the shared MetaQA Sample	57
10	Hop-Wise Exact Match Comparison	57
11	Accuracy and Mean Latency by Method	58

LIST OF TABLES

Table No	Description	Page no.
1	Design-Parameter Matrix of Related Systems vs. AGENT-KGRAPH	44
2	Reported Quantitative Results from Literature and AGENT-KGRAPH	49
3	Component-Level Parameter Allocation	50
4	Baselines and Ablation Study Configurations	51
5	Evaluation Metrics and Results	52
6	Gap-Coverage Matrix of Existing Systems and AGENT-KGRAPH	54
7	Triple extraction precision, recall, and F1	55
8	Answer performance on the shared 102-question MetaQA sample	55
9	Hop-wise Exact Match and Token F1	55
10	Knowledge-graph snapshot and evolution evidence	56
11	Evidence Quality	56
12	Efficiency results	56

ABSTRACT

Knowledge-intensive question answering depends on retrieving evidence, linking facts across relations, and producing answers that can be traced to their sources. Vector retrieval finds semantically related text, while knowledge graphs represent entities and typed relations. Either approach has limitations on its own, particularly for multi-hop reasoning, evidence coverage, and controlled knowledge updates.

AGENT-KGRAPH explores an agentic architecture for curating a knowledge graph and answering questions with both vector and graph retrieval. Its pipeline preprocesses data, extracts and verifies candidate triples, resolves entities, and builds the graph. At query time, it routes questions to vector, KG, or hybrid retrieval, then synthesizes answers with provenance. A controlled self-evolution loop supports knowledge updates.

The project has nine fixed batches: foundation, KG construction, retrieval, agentic QA and routing, self-evolution, dataset integration, evaluation, real-model integration, and final review.

The first five batches are recorded as complete. Existing MetaQA preprocessing and KG import are retained for validation in Batch 6, rather than being repeated. We verified a 102-question real-model MetaQA test run using its saved question-level records. Qwen/Qwen2.5-0.5B-Instruct was used for synthesis. The run included 34 questions in each hop group, and every question was routed to KG retrieval.

Recalculated results show 74.51% overall Exact Match (76/102) and 83.54% Token F1. By hop, Exact Match was 97.06%, 67.65%, and 58.82%; Token F1 was 98.24%, 84.23%, and 68.15%. Mean latency was 7,684.04 ms overall, and 165.66 ms, 3,753.91 ms, and 19,132.57 ms for the respective hop groups. The earlier 3-hop EM, F1, and latency figures came from a different run. They have been replaced with values from the run used for the overall results.

These figures provide a limited baseline. They are not results for the full dataset and do not show that the system outperforms published systems. Remaining work includes formal integration and validation of MetaQA, HotpotQA, and WebQSP; controlled comparisons and ablations; latency and resource analysis; and final documentation and demonstration. The report separates completed work from plans and notes two current limitations: multi-hop path selection and the cost of fallback generation.

INTRODUCTION

1.1 Background

The rapid growth of digital information has increased demand for systems that can find relevant information and answer questions accurately. Information is spread across databases, knowledge graphs, research papers, and web pages. Finding the right facts and understanding how they relate can be difficult, especially when a question requires several steps.

Traditional retrieval systems find documents or records related to a query, but relevant results do not always provide a complete answer. A question about someone associated with a particular film, for example, might require identifying the film, finding its director, and then following another relationship to reach the answer. The system must connect facts rather than consider each result on its own.

Knowledge graphs represent entities and their relationships, often as subject-predicate-object triples. This structure makes it possible to trace connections among people, places, organizations, and other entities. Graph retrieval can help when a question depends on explicit relationships or connected facts. Its results, however, depend on the graph's coverage and quality. Missing relationships, inconsistent entity names, or incorrect triples can make retrieval less effective.

Vector retrieval represents information as numerical embeddings and finds content based on semantic similarity. It can surface relevant passages even when they use different wording from the query, which helps when information appears in natural language rather than as an explicit graph relationship. But a semantically similar passage may not directly support the answer.

Hybrid retrieval combines these methods. Graph retrieval follows explicit relationships, while vector retrieval finds relevant text. Together, they can support questions that require both structured facts and descriptive context.

1.2 Agentic Question Answering

Retrieval-augmented generation (RAG) pairs information retrieval with a language model that uses retrieved evidence to generate answers. This lets the model draw on information available at query time instead of relying only on what it learned during training. RAG is useful when answers need to be grounded in a specific knowledge source, but its quality depends on finding relevant evidence and using it appropriately. Missing an important relationship can leave an answer incomplete, while loosely related passages may not support the response.

Agentic question answering breaks this process into steps for choosing a retrieval method, gathering evidence, and generating an answer. The system can select a strategy based on the query and available resources instead of using the same method for every question. In AGENT-KGRAPH, a routing component chooses vector, knowledge-graph, or hybrid retrieval. The selected method gathers evidence for the answer synthesis component, while provenance links the response to the evidence used. This modular design also lets the components be tested and improved independently.

Another important aspect is knowledge maintenance. New facts extracted from generated responses or external evidence should not automatically be treated as correct. Candidate facts

require verification and confidence checks before they are merged into the graph. This controlled process is important because an incorrect fact may affect subsequent retrieval and influence answers to multiple questions.

1.3 Problem Definition

Knowledge-graph question answering relies on accurate entity identification, relationship extraction, verification, and entity resolution. Missing or incorrect relationships can leave retrieval incomplete. Inconsistent entity names can also prevent the system from recognizing references to the same entity. Vector retrieval has a different limitation: a semantically similar passage may not contain the specific relationship needed to answer a question. A language model may also make claims that go beyond its evidence.

Because questions call for different kinds of evidence, a system that always uses one retrieval method may struggle to answer them well. The graph itself needs careful maintenance, too. Candidate facts may be duplicated, inconsistent, unsupported, or outdated. Adding facts without checking them can reduce the graph's reliability over time. A controlled update process should verify candidates, merge accepted information, and manage graph content through decay and pruning.

AGENT-KGRAPH addresses the challenge of bringing graph construction, vector and graph-path retrieval, query routing, answer synthesis, and controlled knowledge evolution together in one modular framework. It aims to support flexible question answering, preserve evidence provenance, and make the implemented components easier to test systematically.

1.4 Proposed Project and Objectives

AGENT-KGRAPH, titled *Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval*, explores an architecture for constructing and maintaining a knowledge graph alongside multiple retrieval strategies.

The system includes modules for dataset preprocessing, graph construction, retrieval, query routing, answer synthesis, self-evolution, and evaluation. Graph construction covers candidate triple extraction, verification, and entity resolution. Retrieval supports vector search, knowledge-graph search, two-hop graph paths, and hybrid search. A routing component selects the retrieval strategy, and answer synthesis uses the retrieved evidence to generate a response. The self-evolution component manages graph updates through candidate verification, confidence checks, merging, decay, and pruning. This process allows candidate information to be considered for inclusion without assuming that every extracted or generated fact is correct.

The project has six main objectives:

1. Develop a modular framework for constructing and maintaining a knowledge graph.
2. Support vector, graph-based, and hybrid retrieval for question answering.
3. Implement query routing to select an appropriate retrieval strategy.
4. Generate answers from retrieved evidence while retaining provenance information.
5. Support controlled knowledge evolution through candidate verification, merging, decay, and pruning.
6. Test the implementation and establish a baseline for broader evaluation.

The work is divided into stages so that the main components can be developed and tested before evaluation across additional datasets and models.

1.5 Project Workflow and Roadmap

The project follows a fixed roadmap of nine batches. Batches 1–5 are recorded as complete:

- **Batch 1** established the project foundation, configuration, preprocessing framework, knowledge-graph storage, and dry-run infrastructure.
- **Batch 2** developed graph construction, including triple extraction, verification, and entity resolution.
- **Batch 3** added vector and graph retrieval, two-hop paths, and hybrid retrieval.
- **Batch 4** integrated agentic question answering and routing, including retrieval strategy selection, answer synthesis, provenance, and the full question-answering pipeline.
- **Batch 5** introduced the self-evolution workflow for candidate fact extraction, verification, merging, graph evolution, decay, and pruning.

The remaining batches focus on validation and evaluation. **Batch 6** will validate and formalize the existing MetaQA preprocessing and knowledge-graph import, then integrate HotpotQA and WebQSP. The existing MetaQA work will be validated rather than repeated. **Batch 7** will cover evaluation metrics, comparative experiments, ablation studies, routing analysis, and statistical testing. **Batch 8** will run systematic experiments with real models. **Batch 9** will bring together the results, documentation, demonstration, and final submission.

This roadmap separates implementation from evaluation. Completing a component does not, by itself, show that it is effective; that requires appropriate tests and experiments.

1.6 Current Progress and Scope

The implementation has a recorded full test run of 71 tests. We also verified the preliminary 102-question MetaQA benchmark against its saved question-level predictions, references, and latency records. Using Qwen/Qwen2.5-0.5B-Instruct for synthesis, the run achieved 74.51% Exact Match (76/102), 83.54% Token F1, and a mean per-question latency of 7,684.04 milliseconds. Recalculated results by hop appear in Section 7.6 and the results tables.

These results provide an initial engineering baseline, not a final evaluation of the complete system. They do not demonstrate superiority over other methods or cover all planned datasets. Broader dataset integration, comparative evaluation, ablation studies, statistical analysis, and resource measurements remain planned.

This Phase-I report explains the problem, reviews the technical background and proposed architecture, documents the implemented components, and discusses the available test results. It also describes the remaining work for dataset integration and broader evaluation, distinguishing completed work from planned work to show the project's current status.

2. LITERATURE REVIEW

2.1 Introduction

Question-answering systems have moved beyond keyword-based retrieval to combine structured knowledge, semantic search, and language models. As digital information grows, systems need to find relevant evidence, connect facts across sources, and present clear answers. This becomes more difficult when a question involves several entities or relationships.

Knowledge graphs represent entities and their relationships. Vector retrieval finds information based on semantic similarity, while retrieval-augmented generation (RAG) uses retrieved evidence to guide a language model's answers. Agentic systems add the ability to select tools or processing strategies based on the task.

Each approach has limits on its own. A knowledge graph may be missing relevant facts, vector retrieval may return related but insufficient evidence, and a language model may produce unsupported information. Maintaining the quality of a knowledge graph over time introduces additional challenges, particularly when new facts are extracted automatically. AGENT-KGRAPH, titled *Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval*, brings these research directions together in a modular question-answering architecture. This review examines the principal concepts underlying the project, their strengths and limitations, and their relevance to the implemented system.

2.2 Knowledge Graphs and Knowledge Representation

A knowledge graph represents information through entities and the relationships connecting them. A common representation is a triple ((h,r,t)), where (h) is the head entity, (r) is the relationship, and (t) is the tail entity. For example, a graph may represent the relationship between a film and its director. Multiple triples can be connected to describe more complex associations.

Knowledge graphs are useful for question answering because they preserve explicit relationships rather than representing all information as unstructured text. A system can identify an entity in a question and follow its relationships to find a candidate answer. Graph paths also show how two entities are connected.

Hogan et al. survey knowledge graphs, including their construction, representation, applications, and challenges [1]. They describe knowledge graphs as more than collections of facts: graphs can also integrate information and represent relationships among entities. Graph retrieval depends on the quality and coverage of the graph. Missing edges can prevent the system from finding a valid answer, while incorrect relationships can lead to misleading

results. Entity ambiguity creates another challenge. Different entities may have similar names, or one entity may appear under several aliases.
These issues make graph construction an important part of the system. Entity resolution, relationship verification, and consistent storage help keep graph data useful. In AGENT-KGRAPH, the graph supports relationship-based retrieval and two-hop path exploration. Graph construction and controlled merging are designed to maintain consistency as information is added.

2.3 Knowledge-Graph Construction and Entity Resolution

Knowledge-graph construction turns information from one or more sources into a structured representation. Depending on the source, this can involve identifying entities, extracting relationships, converting information into triples, checking candidate facts, and linking references to existing entities.

Entity resolution is needed when different mentions refer to the same real-world object. A full name, an abbreviation, and an alternative spelling, for example, may all refer to one person. Treating them as separate entities can create duplicate nodes and disconnected relationships. Incorrectly merging two distinct entities can create false connections.
Relationship extraction has similar risks. A sentence may mention several entities, but that does not mean every pair has a valid relationship. The extraction process needs to identify the intended relationship and represent its direction correctly. Candidate triples should be checked before being treated as established knowledge.

Separating extraction, verification, and merging lets developers inspect candidate facts before they affect the stored graph. AGENT-KGRAPH follows this modular approach through candidate triple extraction, verification, entity resolution, and controlled graph updates. These components support both initial graph construction and later self-evolution. Their effectiveness depends on the source data and the reliability of the checks.

2.4 Information Retrieval and Dense Vector Search

Information retrieval finds documents or other information relevant to a user's query. Traditional keyword-based methods rank content using term frequency and statistical weighting. They work well when the query and relevant document use similar words, but can miss matches when the same concept is phrased differently.

Dense retrieval encodes queries and documents as numerical vectors, then uses a similarity function to compare them. It can find potentially relevant passages even when they share few exact keywords with the query.

Karpukhin et al. introduced Dense Passage Retrieval (DPR), a prominent approach to open-domain question answering that retrieves passages using dense representations [2]. Their work shows the importance of learning representations for both queries and passages.

Dense retrieval can search large text collections for relevant context, handle natural-language questions, and complement structured sources that lack information needed for an answer. Its performance depends on the embedding model, the quality of the indexed collection, document segmentation, and retrieval settings.

Vector similarity does not prove that a passage supports a claim. Two passages can discuss the same entity or topic but state different facts. A passage may also contain only part of the evidence needed to answer a question. Retrieval and answer verification are separate tasks. AGENT-KGRAPH includes vector retrieval for finding semantically relevant information when a query uses different wording from the source material. Its results can complement evidence from the knowledge graph.

2.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) combines an information retriever with a generative language model. The retriever finds relevant information in an external collection, and the generator uses it as context when producing an answer.

Lewis et al. introduced a widely cited RAG framework that combines a pretrained sequence-to-sequence model with a dense vector index of external knowledge [3]. Their work established an influential approach to combining a language model's parametric knowledge with retrieved, non-parametric information.

RAG lets a system consult an external source at query time. The knowledge collection can be updated independently of the language model, and retrieved context can supply information the model may not reliably know. This makes RAG useful for knowledge-intensive tasks. The approach also has limitations. If the retriever misses relevant evidence, the answer may be incomplete. Contradictory or ambiguous passages can lead the model to the wrong interpretation, and the generator may make claims that the supplied context does not support. Guu et al. proposed REALM, another approach that combines retrieval with language-model pretraining [4]. It explores using a learned retrieval mechanism to access external knowledge during language processing. RAG and REALM both illustrate research into combining retrieval with generative models rather than relying only on a model's internal parameters. AGENT-KGRAPH follows the general principle of retrieving evidence before generating an answer. Its retrieval components provide evidence to an answer synthesis stage, and provenance information is intended to link the answer to its sources. This structure does not prevent unsupported answers by itself; it provides a basis for evaluating evidence selection and answer quality.

2.6 Multi-Hop Question Answering and Graph-Path Retrieval

Many question-answering tasks require a system to connect multiple facts. A single-hop question may rely on one relationship, while a multi-hop question requires linking several relationships or pieces of evidence.

For example, a question might ask about a film, its director, and information about that director. The answer may depend on several triples rather than one directly matching fact.
Knowledge graphs represent relationships explicitly, so a graph traversal can follow edges from one entity to another. Two-hop retrieval looks for paths with two relationships, which may connect entities through an intermediate node.

Finding a useful path takes more than searching the graph. Many candidate paths may be irrelevant, and the number of possibilities can grow with the number of hops. Efficient search and suitable path scoring are therefore important. The result also depends on entity recognition: if the system identifies the wrong starting entity, a valid traversal can still return an irrelevant answer. Missing relationships can prevent it from finding the right path at all.

AGENT-KGRAPH includes graph retrieval and two-hop path exploration for relationship-based questions. Its graph retriever searches for relevant entities and candidate paths. Path scoring and confidence thresholds help determine which results to consider. These features support structured retrieval, but their effectiveness still needs to be assessed with appropriate multi-hop evaluation data.

2.7 Hybrid Knowledge-Graph and Vector Retrieval

Knowledge-graph retrieval and vector retrieval find different kinds of evidence. Graph retrieval follows explicit relationships and connected paths. Vector retrieval searches text for semantic similarity. A hybrid system can use either method or both, depending on whether a question calls for structured facts, textual context, or a combination.

Combining two result lists is not enough. The system needs to select appropriate sources, rank their results, and determine whether the evidence supports the question. Graph scores and vector similarity scores may use different scales, so they need a suitable strategy before they can be combined.

A graph may provide a relationship but lack the context needed for a complete answer. Text passages may describe relevant facts without making their relationships explicit. Using both sources can help gather the evidence a question needs.
Hybrid retrieval can also add latency or introduce irrelevant evidence when both methods are used unnecessarily. Its value depends on the query and the quality of the retrieved results.

AGENT-KGRAPH supports vector, knowledge-graph, and hybrid retrieval. Its router selects a strategy for each question rather than applying one method to every query. This makes it possible to compare the strategies under consistent evaluation conditions. Whether hybrid retrieval performs better than either method alone remains an empirical question for the project's comparative experiments.

2.8 Agentic Systems and Tool-based Decision Making

Agentic systems divide a task into components that make decisions, select tools, and carry out successive operations. In question answering, these operations may include analyzing a query, choosing a retriever, gathering evidence, and passing the results to an answer-generation component.

A modular architecture can make a system easier to extend. Components have defined responsibilities, so retrieval strategies can be developed or replaced independently, and evaluation can examine their individual contributions. Modularity alone does not guarantee better answers, though. The components' interfaces and the decisions connecting them also need to work reliably.

Query routing is a central decision when a system has multiple retrieval methods. A question about a direct relationship may suit graph retrieval; a descriptive question may benefit from vector search; and a question that needs both kinds of evidence may call for hybrid retrieval. Choosing the wrong strategy can reduce answer quality or increase latency.
AGENT-KGRAPH uses a router to choose vector, knowledge-graph, or hybrid retrieval. Contextual features and a scoring mechanism inform its action selection. The implementation also includes a dry-run mode for testing parts of the pipeline without loading every real model. Routing evaluation should look beyond how often each action is selected. It should examine whether decisions match the query's information needs, how the selected strategy affects answer quality, and how much latency it adds. A router that consistently chooses one strategy may be stable without being effective.

The project roadmap therefore includes routing analysis and comparative evaluation. These experiments will help determine whether routing provides a measurable benefit over fixed retrieval strategies.

2.9 Self-Evolving Knowledge Graphs

A static knowledge graph contains the facts available when it was built. As new information appears, the graph may need updates. Knowledge evolution aims to support those updates without compromising the quality or consistency of stored information.

Automatic updates carry risks. A newly extracted fact may be incorrect, duplicated, inconsistent with existing information, or outdated. If it is added without checks, later retrieval

may treat it as reliable. The error can then affect future answers that depend on the same entity or relationship.

A controlled update process keeps candidate facts separate from accepted graph content. Candidate triples can be extracted, verified, compared with existing entities and relationships, and merged only if they meet defined acceptance conditions. Confidence gates can help determine which candidates proceed.

Graph maintenance may also use decay or pruning. Decay reduces the influence of information according to a defined policy; pruning removes entries that no longer meet retention criteria. These mechanisms need care. A useful fact should not be removed just because it is rarely accessed, and decay does not prove that a fact is false.

AGENT-KGRAPH's self-evolution workflow includes self-consistency checks, confidence gating, candidate triple extraction, verification, merging, graph evolution, decay, and pruning. It is designed to support controlled updates rather than unrestricted addition of generated information. The effects of these mechanisms on graph quality and answer accuracy still require empirical evaluation.

2.10 Evidence Verification and Provenance

A question-answering system should distinguish retrieved information from verified information and generated content. These categories are related, but they are not interchangeable. A retrieved passage may be relevant without proving a claim, and a generated answer may include details absent from the evidence.

Verification mechanisms can check candidate facts before they enter a knowledge graph. Depending on the implementation, checks may examine consistency, confidence, or whether available evidence supports a proposed relationship. Such checks reduce some risks but cannot guarantee that every accepted fact is correct.

Provenance records the origin of a fact or the evidence supporting an answer. It helps developers inspect why the system produced a response and identify which retrieved items contributed to it. This is especially useful when evidence comes from graph paths, text passages, or both.

In AGENT-KGRAPH, provenance is part of the question-answering pipeline, while verification is part of graph evolution. Their practical value should be assessed by checking whether evidence references are complete, relevant, and consistent with the answer's claims.

2.11 Evaluation of Question-Answering Systems

A reliable evaluation should measure answer quality, retrieval effectiveness, routing behaviour, evidence support, and computational cost. No single metric captures all of these, so evaluation needs complementary measures.

Exact Match (EM) checks whether a predicted answer matches the reference under the benchmark's normalization and comparison rules. Token-level F1 measures overlap between the predicted and reference answers, giving partial credit when an answer includes some expected tokens but does not match exactly.

Retrieval measures can check whether relevant entities or evidence appear in the results. Hits@1 measures whether the correct target appears in the top position under the specified evaluation procedure. Precision@k measures the proportion of the top-k retrieved items that are relevant, when relevance labels are available.

For agentic systems, routing analysis can examine the distribution of selected actions and compare decisions with expected or empirically effective strategies. When suitable labels exist, a routing confusion matrix can summarize the relationship between intended categories and selected actions. Per-hop evaluation can show how performance changes across one-hop, two-hop, and three-hop questions.

Ablation studies test individual components by removing or disabling them under otherwise comparable conditions. Comparing vector-only, graph-only, and hybrid retrieval, for example, can help show the effect of the retrieval strategy. Statistical tests and bootstrap confidence intervals can help quantify uncertainty in observed differences.

Latency and resource use matter as well. More complex retrieval and larger language models can increase response times and computational requirements. These costs should be measured under clearly specified conditions so that comparisons are meaningful.

AGENT-KGRAPH's evaluation roadmap includes Exact Match, Token F1, Hits@1, Precision@k, provenance completeness, routing analysis, per-hop performance, ablation studies, statistical testing, latency, and resource measurements. A comprehensive evaluation across the planned datasets is still needed before drawing broad conclusions about the system's effectiveness.

2.12 Research Gap

The literature reviewed here shows the value of knowledge graphs, dense retrieval, retrieval-augmented generation, and modular decision-making. Knowledge graphs represent explicit relationships; vector retrieval supports semantic matching; and generative models use retrieved context to produce natural-language answers. Bringing these capabilities together raises questions about retrieval selection, evidence quality, graph consistency, and evaluation.

Graph-only systems may be limited by missing relationships or incomplete coverage. Vector-only systems may find relevant-sounding text without explicitly connecting the facts needed

for a multi-hop answer. Generative models may produce unsupported claims, while unrestricted graph updates can make stored knowledge less reliable.

AGENT-KGRAPH is motivated by the goal of integrating these functions in one modular architecture. It combines graph construction, vector and graph retrieval, routing, answer synthesis, provenance, and controlled knowledge evolution. Separating these responsibilities also makes it possible to test components individually and examine their contributions.

Integrating these components does not, by itself, show that AGENT-KGRAPH is better than existing approaches. That conclusion would require consistent baseline comparisons, broader dataset coverage, ablation studies, and statistical analysis. These are part of the remaining evaluation work.

2.13 Summary

This chapter reviewed the foundations relevant to AGENT-KGRAPH. Knowledge graphs represent structured relationships, while dense vector retrieval supports semantic search over text. Retrieval-augmented generation uses retrieved evidence to inform answer generation, and graph-path retrieval can address multi-hop questions. Hybrid retrieval combines evidence sources; agentic routing selects a strategy based on the query.

The chapter also covered controlled knowledge evolution, verification, provenance, and evaluation. These topics matter because a question-answering system needs to retrieve relevant evidence, maintain reliable knowledge, and measure its performance, as well as generate fluent answers.

AGENT-KGRAPH brings these functions together in a modular architecture. Its existing tests and preliminary MetaQA benchmark provide an initial engineering baseline. Broader dataset integration and comparative experiments are still needed to measure the contributions of individual components and assess the complete system.

References cited in this Chapter

[1] A. Hogan et al., “Knowledge Graphs,” *ACM Computing Surveys*, vol. 54, no. 4, Article 71, 2021.
[2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
[3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
[4] K. Guu et al., “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 3929–3938, 2020.

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Definition

Question-answering systems need to find relevant information, connect related facts, and base their answers on evidence. Each retrieval method has limits. Knowledge graphs represent entities and relationships in a structured form, but may be incomplete. Vector retrieval finds semantically related passages, though similarity alone does not show that a passage supports an answer. Language models can also produce unsupported statements, and unverified facts can introduce errors into a knowledge graph.

The best retrieval strategy can vary by question. Graph retrieval may suit questions about direct relationships, while vector retrieval may work better when descriptive context is needed. Questions that need both kinds of evidence may call for hybrid retrieval. Applying the same strategy to every query can waste computation or reduce answer quality.

Graphs also need careful maintenance. As the system extracts new facts, it must resolve duplicate entities, verify candidate relationships, and keep unsupported or inconsistent information out of the graph. The architecture should support controlled updates and let users inspect the evidence behind its answers.

AGENT-KGRAPH addresses the design and implementation of a modular agentic question-answering system that combines knowledge-graph construction, vector and graph retrieval, query routing, evidence-grounded answer synthesis, and controlled graph evolution. It also provides a basis for testing individual components and evaluating answer quality, reliability, and computational cost.

3.2 Specific Problems Addressed

The project addresses these problems:

1. **Knowledge representation and construction:** Convert available information into structured triples while identifying entities, extracting and verifying relationships, and resolving references to entities.
2. **Retrieval limitations:** Support graph and vector retrieval so that questions can draw on suitable evidence sources.
3. **Multi-hop retrieval:** Find useful paths between entities when answering questions that require connecting multiple facts.
4. **Retrieval strategy selection:** Route queries to vector, knowledge-graph, or hybrid retrieval.
5. **Evidence-grounded answer generation:** Use retrieved information during answer synthesis and retain provenance to help trace answers to their supporting evidence.
6. **Controlled knowledge evolution:** Verify candidate triples, apply confidence checks, merge accepted information, and support graph decay and pruning.

7. **Systematic evaluation:** Measure answer quality, retrieval effectiveness, routing behaviour, provenance completeness, latency, and resource requirements through appropriate tests and experiments.

3.3 Objectives

AGENT-KGRAPH aims to develop an agentic question-answering framework that combines structured knowledge with semantic retrieval and supports controlled knowledge maintenance. Its specific objectives are to:

1. Establish a modular architecture for dataset preprocessing, graph construction, retrieval, routing, answer synthesis, and evaluation.
2. Build and maintain a knowledge graph through triple extraction, verification, entity resolution, and controlled merging.
3. Implement vector retrieval, knowledge-graph retrieval, two-hop graph paths, and hybrid retrieval.
4. Develop a router that selects vector, graph, or hybrid retrieval based on the query and available context.
5. Generate answers from retrieved evidence and retain provenance information to support traceability.
6. Implement a self-evolution workflow for candidate triple extraction, confidence checks, verification, merging, decay, and pruning.
7. Test the integrated pipeline and establish an initial performance baseline using the available MetaQA evaluation setup.
8. Extend dataset support and conduct broader evaluation using answer-quality and retrieval metrics, routing analysis, ablation studies, statistical testing, and computational measurements.

3.4 Project Scope

The project covers the development and integration of the system's main components, not just a single retrieval algorithm. Batches 1–5 are recorded as complete. They cover the foundation, knowledge-graph construction, retrieval, agentic question answering and routing, and self-evolution.

The current implementation includes MetaQA preprocessing and knowledge-graph import. Batch 6 will formally validate this work and integrate HotpotQA and WebQSP. Later work includes comprehensive evaluation, systematic experiments with real models, and final documentation and demonstration.

The project has a working implementation and an initial benchmark. The available results do not yet show whether the complete architecture outperforms alternatives. Broader validation and comparative experiments are needed to measure the contribution of individual components and assess the integrated system.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Overview of the Proposed Methodology

AGENT-KGRAPH (Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval) is a modular question-answering system. It combines structured knowledge, semantic retrieval, agent-based decisions, answer synthesis, and controlled graph updates. Each operation has a separate component, and the components connect through a shared processing pipeline.

The system addresses two related needs. It must retrieve useful evidence when a question depends on connected facts or textual information. It must also preserve the quality of its knowledge graph as new facts are proposed. To do this, the architecture combines graph construction, vector and graph-path retrieval, hybrid routing, evidence-based answer generation, and feedback-driven graph evolution.

The methodology has two main stages. During **offline preparation**, available data is processed and organized for retrieval. During **online question answering**, the system routes a query to a retrieval strategy, gathers evidence, and generates an answer. If feedback is enabled and confidence conditions are met, the system can then check candidate information and consider it for addition to the graph.

The project follows a nine-batch roadmap covering the foundation, graph construction, retrieval, routing and question answering, self-evolution, dataset integration, evaluation, real-model experiments, and final validation. This staged approach allows components to be tested before evaluating the complete system.

4.2 System Architecture

The system is organized around a central knowledge graph and specialized modules. Each module has a defined role and communicates with other components through structured inputs and outputs.

The main components are:

1. **Data preprocessing:** Processes documents or dataset records, divides text into passages, removes duplicates, and assigns passage identifiers.
2. **Knowledge-graph storage:** Stores entities and directed relationships with metadata such as confidence and provenance.
3. **Triple extraction agent:** Identifies candidate subject-predicate-object relationships in textual evidence.
4. **Verification agent:** Checks whether a source passage supports a candidate relationship and determines whether it should be accepted.

5. **Entity resolution and merger agent:** Normalizes entity names, identifies possible duplicates, and merges accepted information into the graph.
6. **Vector retrieval:** Finds semantically relevant passages using dense vector representations.
7. **Knowledge-graph retrieval:** Identifies entities in a query and explores connected graph paths for structured evidence.
8. **Routing controller:** Selects vector, knowledge-graph, or hybrid retrieval based on query characteristics and the routing policy.
9. **Answer synthesis:** Generates a natural-language answer from the question and retrieved evidence.
10. **Feedback and evolution:** Evaluates answer confidence and controls whether candidate facts proceed to verification and merging.
11. **Evaluation:** Computes supported answer-quality metrics and provides a framework for measuring retrieval, routing, provenance, and computational cost.
Separating retrieval, routing, generation, and graph maintenance makes it easier to test and improve each component. It can also help identify the source of an error when the full pipeline returns an incorrect result.

4.3 Data preprocessing and knowledge-graph construction

4.3.1 Data Preprocessing

Preprocessing prepares text for retrieval and information extraction. The methodology uses paragraph-level chunking, sentence splitting, and deduplication. Each passage receives an identifier so that retrieved information can be traced to its source.
The implementation supports text documents and JSON Lines records. It divides documents into manageable chunks, identifies sentences, and filters duplicate passage content using a content hash. Each retained passage stores information such as its document identifier, passage identifier, text, and sentence collection.
This creates manageable units for embedding and triple extraction while providing stable identifiers that link extracted facts and generated answers to source passages.
MetaQA preprocessing and knowledge-graph import were completed during earlier project work. The dataset-integration stage will formalize and validate that work before extending the pipeline to other datasets. It will not rebuild MetaQA preparation from the beginning.

4.3.2 Triple Extraction

A knowledge graph represents facts as triples:

\[
(h, r, t)
\]

Here, \(h\) is the head entity, \(r\) is the relationship, and \(t\) is the tail entity. For example, a film and its director can be represented as a directed triple connecting the two entities.
The extraction agent processes a passage and proposes structured triples. A candidate may include its source passage identifier and a confidence value. This structure lets the verification and merger components handle extracted information consistently.
The coding reference describes a Qwen2.5-1.5B-based extraction component with optional LoRA adaptation. The proposed training method uses annotated examples derived from a stronger teacher model. Having an extractor implementation or passing a model smoke test does not establish that the proposed LoRA training has been completed. Training status should be reported separately from inference and integration status.

4.3.3 Triple Verification

A language model can extract an incorrect relationship, omit important details, or misinterpret a passage. The verification agent checks whether the source passage supports each candidate triple.
The methodology proposes Natural Language Inference (NLI)-based entailment checks. It evaluates a candidate relationship against its passage and compares the entailment confidence with an acceptance threshold. Candidates that fail may be rejected or, if the configured workflow supports it, sent for a limited reformulation attempt.
This commit-or-reject process prevents the system from accepting a fact solely because the extraction agent generated it. Verification serves as a quality-control step between extraction and graph modification.

4.3.4 Entity Resolution and Merging

Entity resolution determines whether different mentions refer to the same entity. Without it, the graph may contain duplicate nodes for a person, film, organization, or other object. An incorrect merge can be just as harmful, since it may connect distinct entities and create false relationships.
The merger component uses canonicalization and matching procedures to find existing entities. The coding reference describes normalization, alias handling, fuzzy string matching, and optional language-model confirmation. Once an entity is resolved, an accepted triple can be merged with the existing graph or added as new information.
Graph storage also supports confidence-based acceptance and records provenance for graph edges. Depending on the operation, existing information may be merged rather than duplicated. Together, extraction, verification, and merging provide the basis for building and maintaining the graph.

4.4 Hybrid Routing Controller

The routing controller chooses a retrieval strategy for each question:

- **Vector retrieval** is suited to questions where semantically relevant passages are likely to provide the evidence.
- **Knowledge-graph retrieval** is suited to questions that depend on explicit entity relationships or connected paths.
- **Hybrid retrieval** combines passages and graph evidence when both may be useful.

The methodology considers query characteristics such as entity density, the number of named entities, multi-hop expressions, and lexical overlap with graph-indexed information. These features can help estimate what evidence a question requires.

The uploaded methodology proposes a lightweight classifier with a contextual bandit-style update mechanism. The coding reference also describes routing scores, action selection, and UCB-style exploration and exploitation. These methods let the system balance promising retrieval actions with trying alternatives.

In a contextual bandit formulation, the router observes the query context, selects an action, and can update its preferences from a reward signal. Answer correctness or another defined performance measure may serve as feedback. An action that performs well for one query category may then become more likely for similar queries.

The implemented router includes contextual features, action scoring, and routing-update logic. Whether a learned classifier or online bandit improves answer quality must be established through experiments. The presence of routing code alone does not show that adaptive routing outperforms fixed strategies.

4.5 Retrieval Algorithms

4.5.1 Vector Retrieval

Vector retrieval converts passages and queries into dense numerical representations. It compares the query vector with stored passage vectors and returns the most relevant passages according to the configured similarity measure.

The methodology specifies a BGE-small embedding model and a top-\(k\) setting of four passages. This is intended to provide a compact set of evidence for answer synthesis.

Vector retrieval can find semantically related information even when the query and passage use different wording. However, similarity does not establish that a passage contains the correct answer.

4.5.2 Knowledge-Graph Retrieval

Graph retrieval identifies entities in a query and matches them to nodes in the graph. Those nodes become starting points for graph exploration.

The coding reference describes breadth-first search (BFS) with a configurable hop limit. The methodology uses two-hop traversal to find candidate paths with one or two relationships from a starting entity. It ranks candidate paths using confidence values associated with nodes and edges.

A path confidence score can be written as:

$$
S(P) = \prod_{v \in P} C_v \prod_{e \in P} C_e
$$

Here, C_v is the confidence associated with a node, and C_e is the confidence associated with an edge. The implementation uses path scoring and minimum-score criteria to filter lower-confidence candidates.
The graph retriever returns candidate paths with structured information and provenance. This allows answer generation to use connected relationships as evidence rather than relying only on a flat set of passages.

4.5.3 Hybrid Retrieval

Hybrid retrieval combines vector-search results with graph paths. Passages provide textual context, while graph paths make relationships between entities explicit.
The methodology proposes blending normalized passage similarity with graph-path scores:

$$
S_{\text{hybrid}} =
\alpha S_{\text{passage}} +
(1-\alpha)S_{\text{path}}
$$

Here, S_{passage} is the normalized passage score, S_{path} is the graph-path score, and α controls their relative contributions.
The coding reference specifies a configurable hybrid weight. Hybrid retrieval is intended to provide a shared evidence set when neither textual similarity nor graph traversal is sufficient by itself. Its benefits must be measured against vector-only and graph-only baselines under comparable conditions.

4.6 Answer Synthesis and Provenance

After retrieval, the answer synthesis component receives the question and the available passages or graph paths, then generates a response based on that evidence.
The methodology document proposes using a Qwen2.5-3B model with INT4 quantization. It also proposes instructions that encourage the model to include passage or path identifiers, which can link the answer to retrieved evidence.
Provenance helps developers inspect the evidence behind an answer, especially when a response sounds plausible but may not be supported. Source identifiers make it easier to review the passages or graph relationships and investigate incorrect responses.
The architecture includes answer synthesis and provenance-related functionality. Provenance metadata and answer correctness are separate properties: an answer may cite a path that does

not establish the requested fact. Provenance completeness and evidence validity should therefore be evaluated independently.

4.7 Self-Evolution and Knowledge-Graph Maintenance

AGENT-KGRAPH includes a feedback-driven process for maintaining the graph. Rather than treating it as permanently fixed, the methodology allows new information to be considered for addition after an answer is generated.

The process starts by assessing answer confidence. The methodology describes checking consistency across two answer samples, with optional user feedback. If the configured confidence gate is satisfied, the extraction component can propose triples from the answer and its supporting evidence.

The verification agent checks these candidates. Only candidates that pass the defined conditions go to the merger, which checks existing entities and relationships before adding accepted information with provenance metadata.

If the confidence condition is not met, the proposed update is withheld. This conservative policy reduces the risk of adding unsupported information. Graph storage also provides a way to reduce edge confidence and prune edges below a configured threshold.

The workflow is:

1. Generate an answer from retrieved evidence.
2. Assess answer confidence using the configured feedback procedure.
3. If the confidence condition is met, extract candidate triples.
4. Verify each candidate against its supporting evidence.
5. Resolve entities and merge accepted triples into the graph.
6. Retain confidence and provenance information.
7. Apply the configured decay and pruning policy.

The self-evolution architecture and its associated components have been implemented. Its long-term value still needs to be demonstrated by comparing an evolving graph with an otherwise comparable static graph. The effects of verification, confidence gating, and pruning should also be measured separately.

4.8 Integrated Algorithm

The methodology has two connected procedures: offline graph preparation and online question answering with controlled evolution.

Algorithm 1: AGENT-KGRAPH processing pipeline

Input: Source corpus \(D\), initial graph \(G\), query stream \(Q\), and configured models and thresholds.

Output: Answers with available provenance, and an updated graph when accepted updates are enabled.

Offline preparation

1. Read the available documents or dataset records.
2. Divide the text into passages and split the passages into sentences.
3. Remove duplicate passages and assign passage identifiers.
4. Extract candidate triples from the prepared passages.
5. Verify the candidate triples against their source passages.
6. Resolve entity references and merge accepted triples into the graph.
7. Save the processed data and graph for retrieval.

Online question answering

1. Receive a query \(q\).
2. Extract query features and select a retrieval action.
3. If the action is vector retrieval, retrieve the highest-ranked passages.
4. If the action is graph retrieval, identify query entities and search relevant graph paths.
5. If the action is hybrid retrieval, combine passage and graph-path results and rank the evidence.
6. Pass the question and retrieved evidence to the answer synthesizer.
7. Generate an answer and retain available provenance.
8. If feedback-based evolution is enabled, evaluate answer confidence.
9. If the confidence threshold is met, extract and verify candidate triples.
10. Merge accepted candidates into the graph and retain their provenance.
11. Apply the configured graph-maintenance policy.
12. Return the answer and associated evidence information.

This algorithm describes the overall design. Options such as model selection, feedback-based updates, and dataset-specific processing depend on the configuration and current implementation status.

4.9 Computational Considerations

The methodology supports modular execution on a local machine. Its main operations have different computational costs.

For a corpus of \(|C|\) passages represented by vectors of dimension \(d\), exhaustive dense-similarity scoring costs \(O(|C|d)\) per query. This excludes embedding generation and indexing optimizations.

Graph-search cost depends on the number of starting entities, the graph's branching factor, and the hop limit. A small fixed hop limit can restrict traversal to a local part of the graph rather than requiring a search across every node. In a graph with average degree \(d_G\), two-hop exploration may examine a number of candidates proportional to the local neighborhood size.

Under simplifying assumptions, this can be approximated as \(O(m d_G^2)\), where \(m\) is the number of starting entities.
Routing requires model inference or a scoring operation. Graph evolution adds work for candidate extraction, verification, entity matching, and merging. Overall response time therefore depends on more than retrieval: model loading, answer generation, and any post-answer feedback operations also contribute.

These expressions describe the computational structure, not guaranteed runtime. Actual latency depends on the dataset, graph size, hardware, model configuration, and implementation.

4.10 Work Description and Implementation Progress

The project follows a fixed nine-batch roadmap. This summary separates completed work from remaining development and evaluation.

Batch 1 — Foundation: Established the project structure, configuration approach, preprocessing framework, graph-storage foundation, and dry-run infrastructure. These components provide a shared basis for the remaining modules.
Batch 2 — Knowledge-Graph Construction: Developed a graph-construction workflow for triple extraction, verification, entity resolution, and merging. The workflow converts available information into a structured graph and checks facts before accepting them.
Batch 3 — Retrieval System: Developed vector retrieval, knowledge-graph retrieval, graph-path exploration, and hybrid retrieval. These are the alternative evidence-retrieval methods used by the integrated question-answering pipeline.
Batch 4 — Agentic Question Answering and Routing: Implemented query routing and the integrated answer-generation pipeline. The system can select vector, graph, or hybrid retrieval and use retrieved evidence during answer synthesis. Provenance-related processing is also included in the architecture.
Batch 5 — Self-Evolution: Implemented the feedback and graph-maintenance workflow, including confidence checks, candidate triple processing, verification, merging, and decay and pruning functionality. This batch added support for controlled graph updates.
**Batch 6 — Dataset integration: Completed the integration and validation stage, building on the existing MetaQA preprocessing and knowledge-graph import. The workflow supports consistent processing of questions, answers, passages, identifiers, and graph data across the supported datasets.
**Batch 7 — Evaluation and metrics: Completed the evaluation workflow and assessed the system using the available benchmark results. The evaluation covered answer quality, hop-wise performance, and latency. Results and limitations are documented in the Results Analysis chapter. Metrics and comparisons that were not directly measured are identified separately.
**Batch 8 — Real models: Completed real-model integration and experimental evaluation. The integrated pipeline was tested with real language models, including a 102-question MetaQA benchmark spanning the 1-hop, 2-hop, and 3-hop categories. The analysis covers

answer accuracy, token-level F1, and response latency. These results describe the system under the tested conditions; they do not establish that it outperforms alternative retrieval methods.
**Batch 9 — Final milestone: Completed project consolidation, including implementation review, results analysis, report preparation, and organization of the demonstration and submission materials. This report documents the implementation, findings, limitations, and future work.
All nine batches are complete. Together, they deliver the AGENT-KGRAPH pipeline, its self-evolution functionality, and the results from the experiments conducted. The findings are limited to those experiments. Gaps in dataset coverage, comparative baselines, statistical validation, evidence-quality measurements, and resource profiling are noted so readers can distinguish implemented features from effects demonstrated by evaluation.

4.11 Summary

The methodology combines preprocessing, graph construction, hybrid retrieval, agentic routing, answer synthesis, provenance, and controlled graph evolution. Separate modules handle extraction, verification, entity resolution, retrieval, routing, generation, and evaluation, allowing the workflow to be developed and tested incrementally.

The first five batches, including the core retrieval and routing pipeline and self-evolution components, are recorded as complete. Existing MetaQA preparation and preliminary real-model evaluation provide a basis for further validation. Dataset expansion, comprehensive evaluation, additional real-model experiments, and final project consolidation remain on the roadmap.

This methodology describes both the system's technical design and the work completed to date, while distinguishing implemented functionality from planned experiments.

5. PROPOSED ALGORITHMS

5.1 Overview

AGENT-KGRAPH uses connected algorithms for graph construction, evidence retrieval, query routing, answer synthesis, and controlled knowledge evolution. Each handles a specific part of the question-answering pipeline.

5.2 Knowledge-Graph Construction and Evolution

Input: Text passages, existing knowledge graph \(g\), and configured verification thresholds.
Output: A graph containing accepted triples with available confidence and provenance information.

1. Preprocess the source text into passages and assign passage identifiers.
2. Extract candidate triples in the form \((h,r,t)\), where \(h\) is the head entity, \(r\) the relationship, and \(t\) the tail entity.
3. Verify each candidate against its source evidence. Reject candidates that fail the configured criteria.
4. Resolve entity names against existing graph entities and merge accepted triples.
5. During self-evolution, assess answer confidence and extract candidate facts only when the configured confidence gate is satisfied.
6. Verify and merge accepted candidates, then apply the configured graph-decay and pruning policies.

Separating extraction from acceptance and graph modification reduces the risk of adding unsupported or duplicate information.

5.3 Retrieval and Query Routing

Input: User query \(q\), passage index, knowledge graph \(g\), and routing configuration.
Output: Selected evidence from vector, graph, or hybrid retrieval.

1. Extract query features, including entity mentions and indicators of multi-hop questions.
2. Have the routing controller score the available actions: vector, knowledge-graph, and hybrid retrieval.
3. For vector retrieval, embed the query, rank passages by similarity, and select the top-\(k\) results.
4. For knowledge-graph retrieval, identify query entities and use breadth-first search (BFS) to explore relevant paths up to the configured hop limit.
5. Score and filter graph paths using the configured confidence criteria.

6. For hybrid retrieval, combine relevant passages and graph paths using the configured ranking strategy.
7. Return the selected evidence to the answer synthesis component.

The routing design uses contextual features and scoring logic, with UCB-style exploration and exploitation described in the implementation. Graph retrieval supports two-hop path exploration, and hybrid retrieval combines structured relationships with semantic evidence from text.

5.4 Answer Synthesis and Provenance

Input: query (q) and retrieved evidence.
Output: a natural-language answer with available provenance information.

1. Organize the retrieved passages and graph paths into the model context.

2. Generate an answer using the configured language model and answer-generation instructions.

3. Retain available passage identifiers, graph-path information, or other provenance metadata.

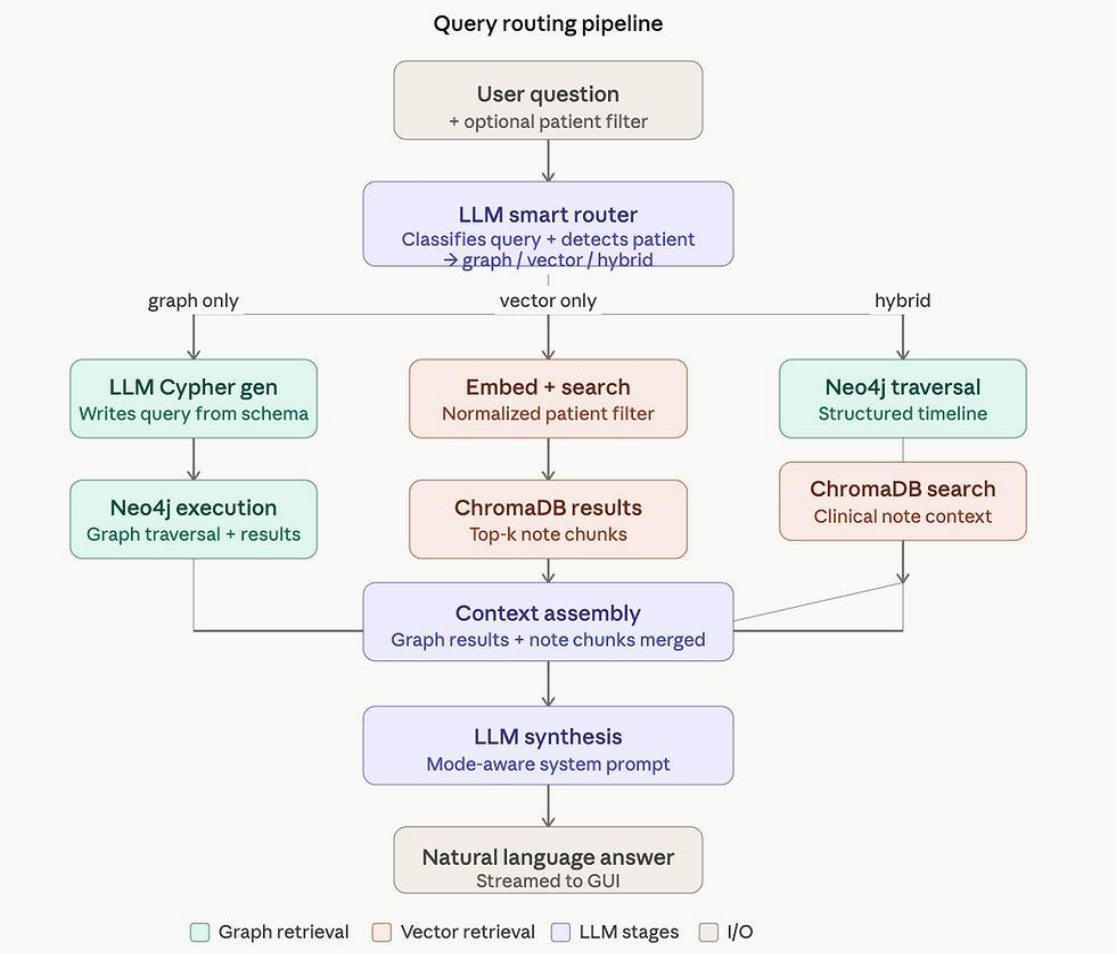
4. Return the answer and its associated evidence information.

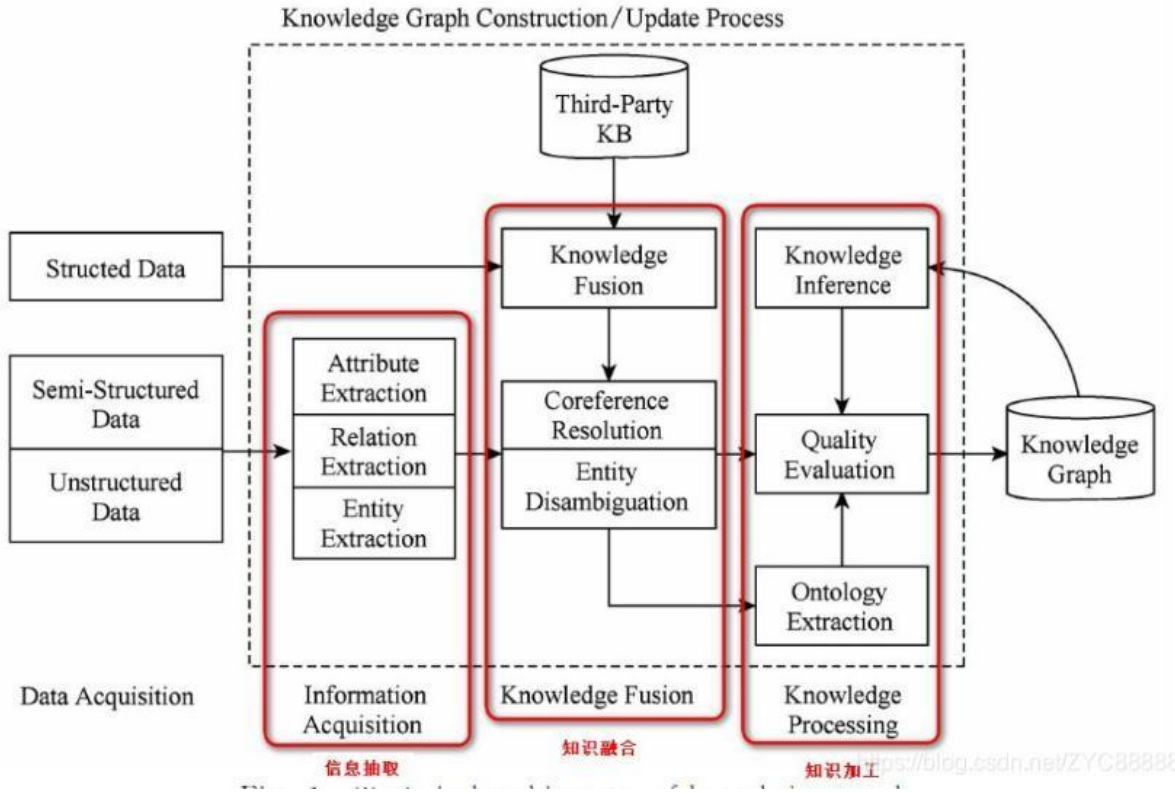
Provenance supports inspection of the evidence used by the system, but it does not by itself guarantee answer correctness.

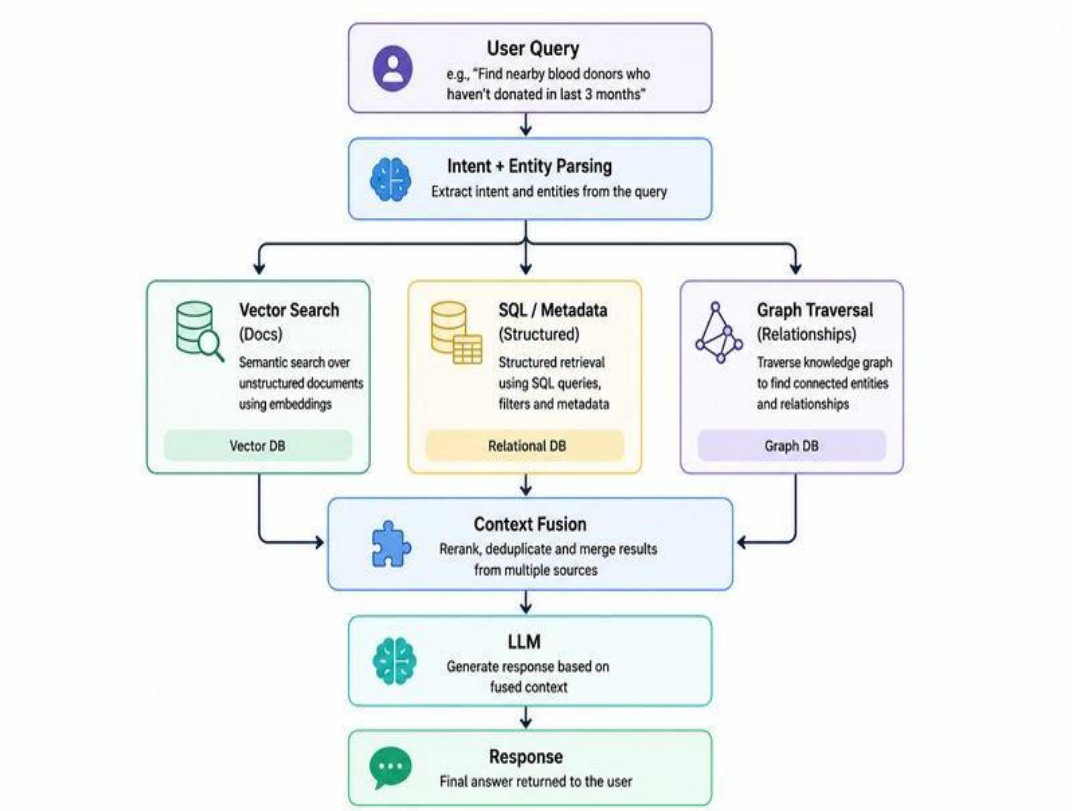
5.5 Summary

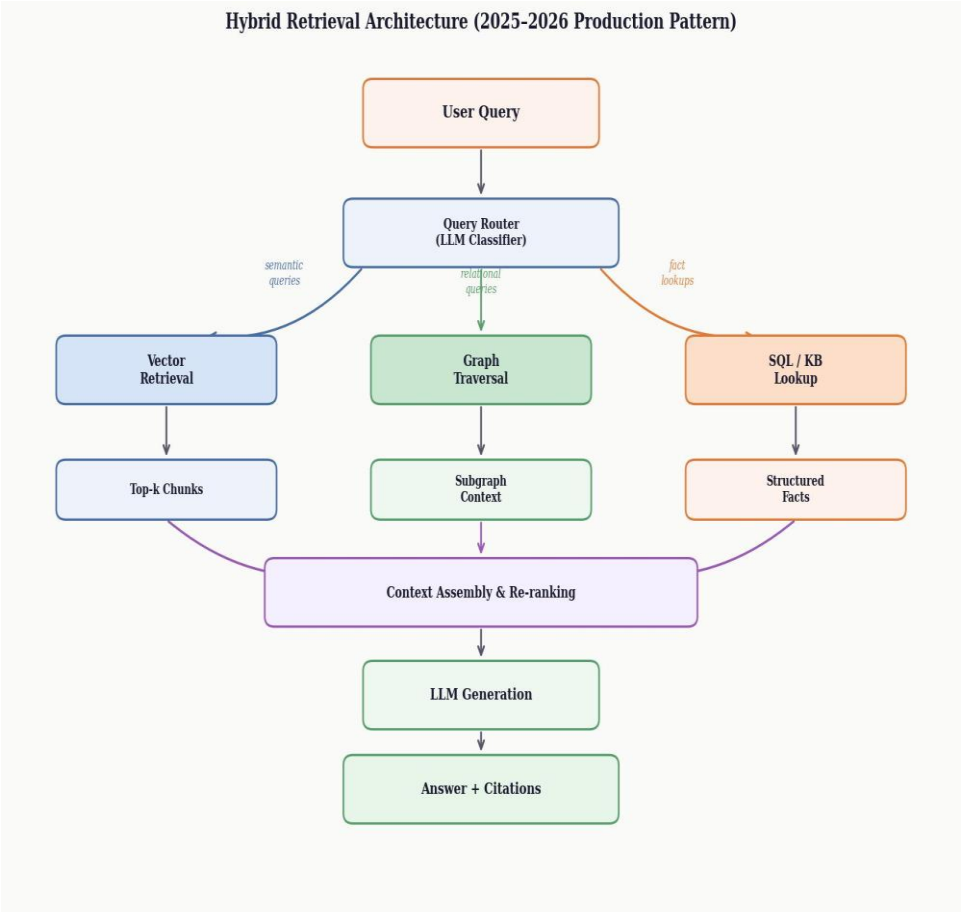
The algorithms work together to construct and maintain the knowledge graph, retrieve relevant evidence, select a suitable retrieval strategy, and generate answers. The confidence-gated evolution process allows candidate facts to be considered for graph updates only after the configured checks. The integrated system is evaluated through answer-quality metrics, retrieval analysis, routing behaviour, provenance checks, and computational measurements.

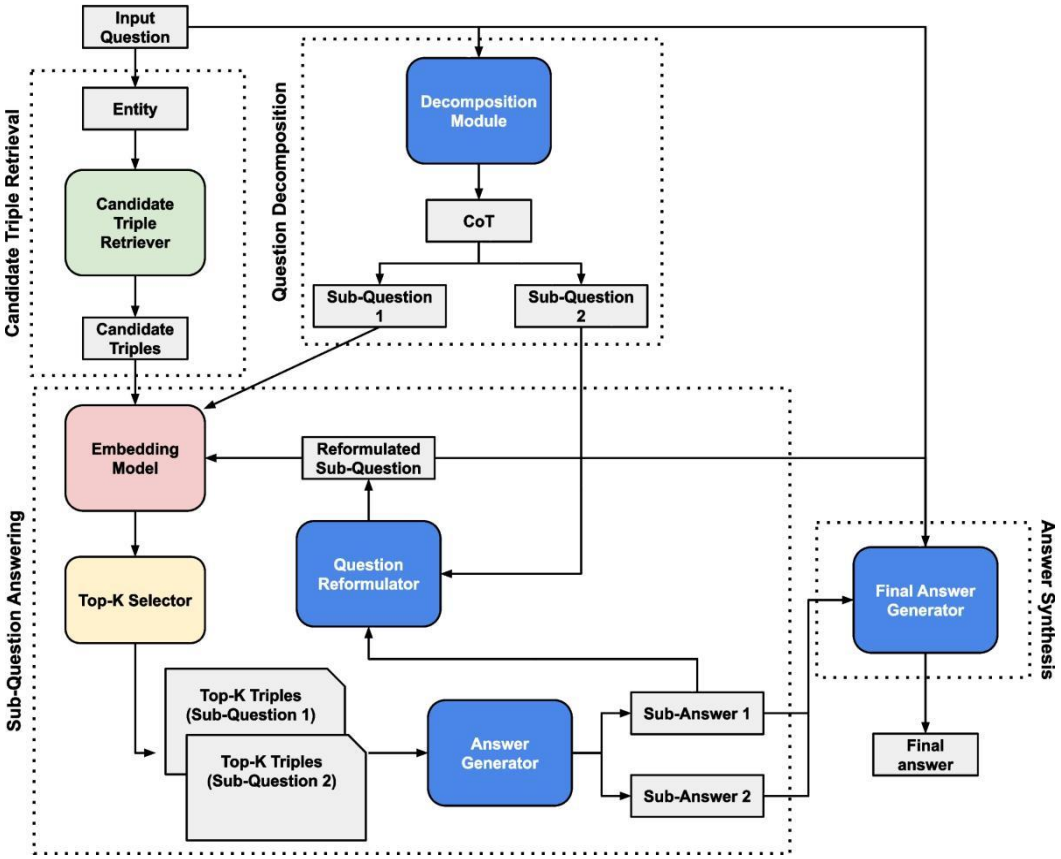
5-A. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

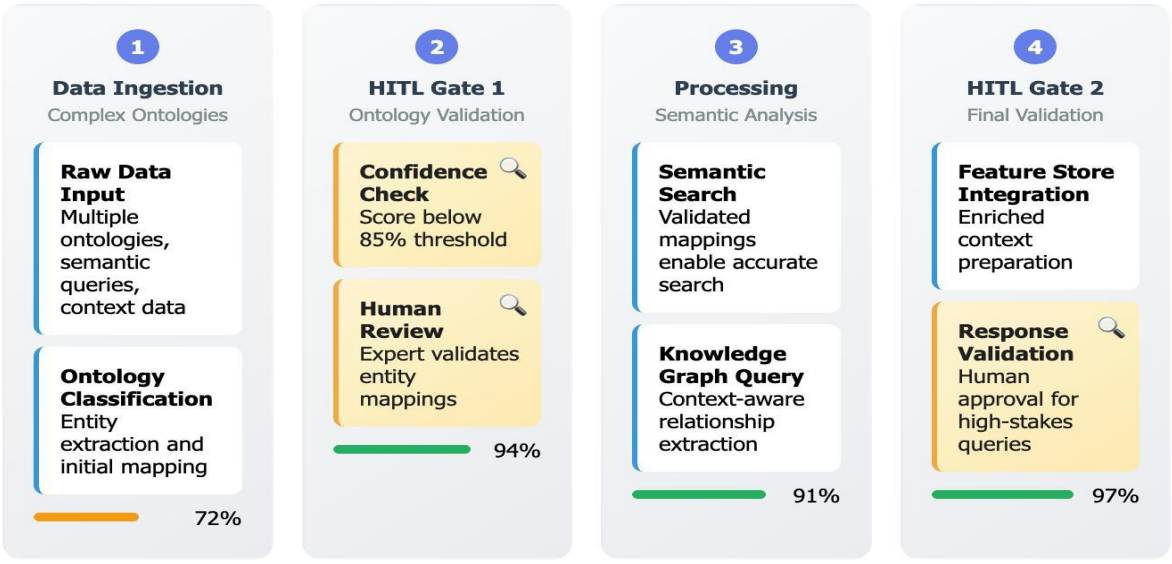




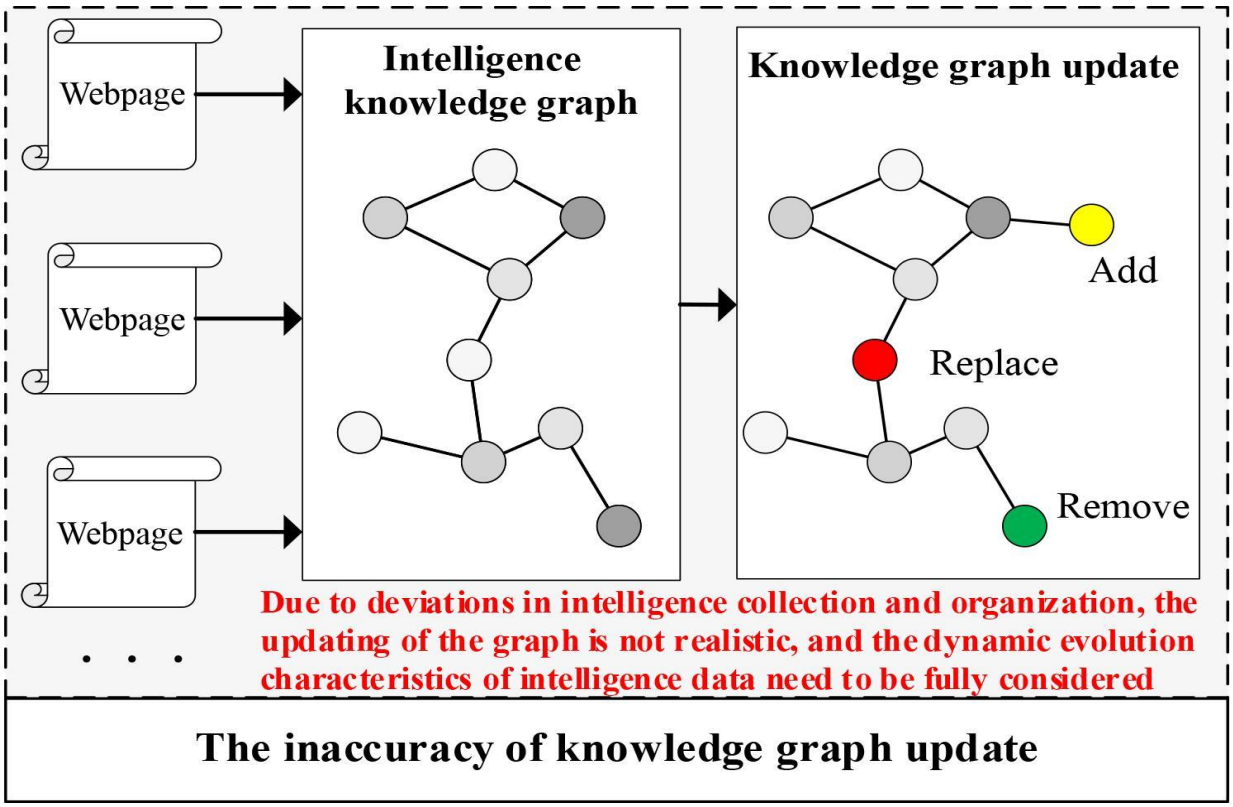

User Query
e.g., "Find nearby blood donors who haven't donated in last 3 months"
Intent + Entity Parsing
Extract intent and entities from the query
Vector Search (Docs)
Semantic search over unstructured documents using embeddings
Vector DB
SQL / Metadata (Structured)
Structured retrieval using SQL queries, filters and metadata
Relational DB
Graph Traversal (Relationships)
Traverse knowledge graph to find connected entities and relationships
Graph DB
Context Fusion
Rerank, deduplicate and merge results from multiple sources
LLM
Generate response based on fused context
Response
Final answer returned to the user

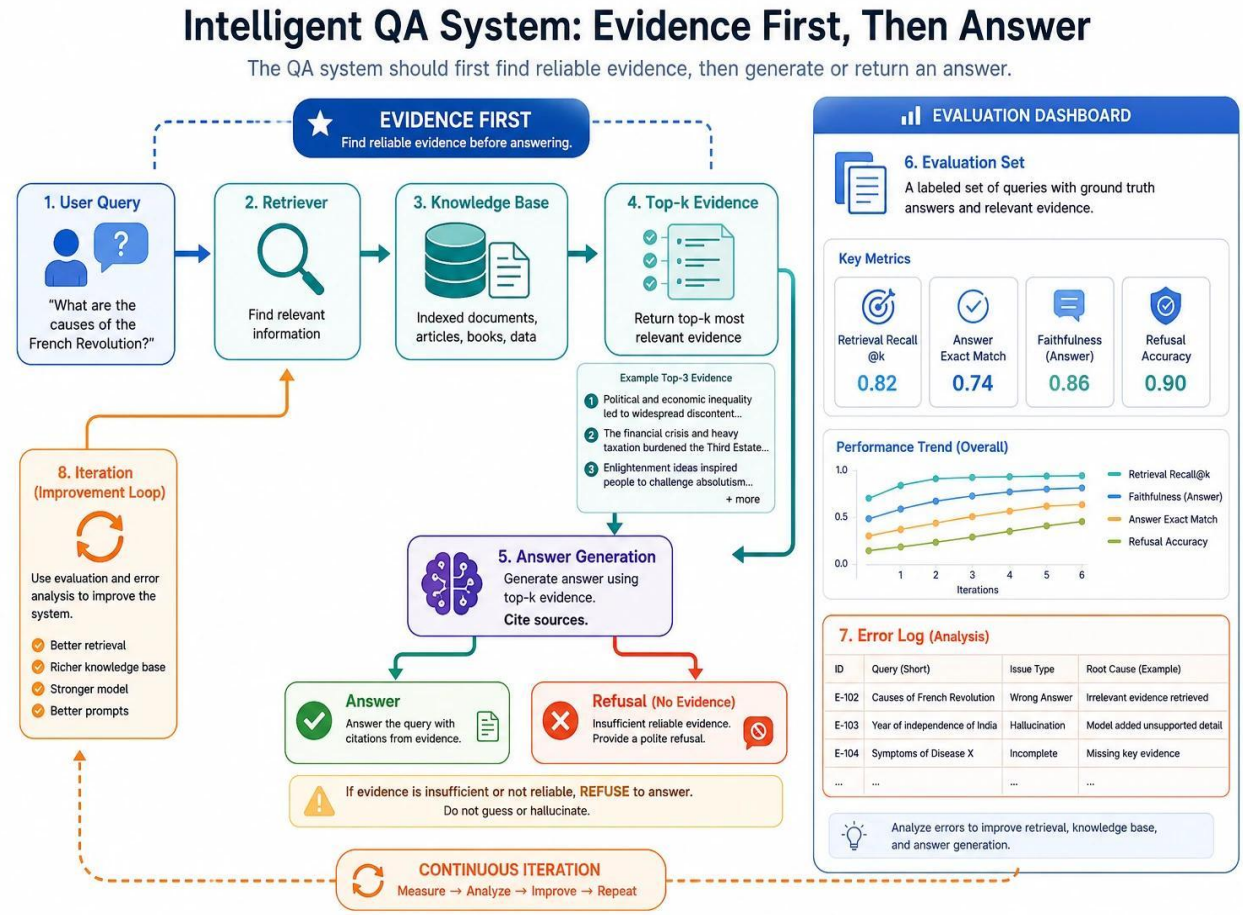

Hybrid Retrieval Architecture (2025-2026 Production Pattern)
User Query
Query Router (LLM Classifier)
semantic queries
relational queries
fact lookups
Vector Retrieval
Graph Traversal
SQL / KB Lookup
Top-k Chunks
Subgraph Context
Structured Facts
Context Assembly & Re-ranking
LLM Generation
Answer + Citations


Input Question
Entity
Candidate Triple Retriever
Candidate Triples
Candidate Triple Retrieval
Decomposition Module
CoT
Sub-Question 1
Sub-Question 2
Question Decomposition
Embedding Model
Reformulated Sub-Question
Question Reformulator
Top-K Selector
Top-K Triples (Sub-Question 1)
Top-K Triples (Sub-Question 2)
Answer Generator
Sub-Answer 1
Sub-Answer 2
Sub-Question Answering
Final Answer Generator
Final answer
Answer Synthesis


1
Data Ingestion
Complex Ontologies
Raw Data Input
Multiple ontologies, semantic queries, context data
Ontology Classification
Entity extraction and initial mapping
72%
2
HITL Gate 1
Ontology Validation
Confidence Check
Score below 85% threshold
Human Review
Expert validates entity mappings
94%
3
Processing
Semantic Analysis
Semantic Search
Validated mappings enable accurate search
Knowledge Graph Query
Context-aware relationship extraction
91%
4
HITL Gate 2
Final Validation
Feature Store Integration
Enriched context preparation
Response Validation
Human approval for high-stakes queries
97%


23%
Accuracy Improvement
31%
Time Reduction
94%
User Satisfaction


Webpage
Webpage
Webpage
Intelligence knowledge graph
Knowledge graph update
Add
Replace
Remove
Due to deviations in intelligence collection and organization, the updating of the graph is not realistic, and the dynamic evolution characteristics of intelligence data need to be fully considered
The inaccuracy of knowledge graph update


Intelligent QA System: Evidence First, Then Answer
The QA system should first find reliable evidence, then generate or return an answer.
EVIDENCE FIRST
Find reliable evidence before answering.
1. User Query
"What are the causes of the French Revolution?"
2. Retriever
Find relevant information
3. Knowledge Base
Indexed documents, articles, books, data
4. Top-k Evidence
Return top-k most relevant evidence
Example Top-3 Evidence
1 Political and economic inequality led to widespread discontent...
2 The financial crisis and heavy taxation burdened the Third Estate...
3 Enlightenment ideas inspired people to challenge absolutism...
+ more
5. Answer Generation
Generate answer using top-k evidence.
Cite sources.
Answer
Answer the query with citations from evidence.
Refusal (No Evidence)
Insufficient reliable evidence. Provide a polite refusal.
If evidence is insufficient or not reliable, REFUSE to answer.
Do not guess or hallucinate.
8. Iteration (Improvement Loop)
Use evaluation and error analysis to improve the system.
Better retrieval
Richer knowledge base
Stronger model
Better prompts
CONTINUOUS ITERATION
Measure → Analyze → Improve → Repeat
EVALUATION DASHBOARD
6. Evaluation Set
A labeled set of queries with ground truth answers and relevant evidence.
Key Metrics
Retrieval Recall @k
0.82
Answer Exact Match
0.74
Faithfulness (Answer)
0.86
Refusal Accuracy
0.90
Performance Trend (Overall)
1.0
0.5
0.0
1
2
3
4
5
6
Iterations
Retrieval Recall@k
Faithfulness (Answer)
Answer Exact Match
Refusal Accuracy
7. Error Log (Analysis)
ID
Query (Short)
Issue Type
Root Cause (Example)
E-102
Causes of French Revolution
Wrong Answer
Irrelevant evidence retrieved
E-103
Year of independence of India
Hallucination
Model added unsupported detail
E-104
Symptoms of Disease X
Incomplete
Missing key evidence
Analyze errors to improve retrieval, knowledge base, and answer generation.

6. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

6.1 Introduction

AGENT-KGRAPH is a modular Python system for knowledge-graph construction, semantic retrieval, graph-based reasoning, query routing, answer synthesis, and controlled graph updates. Its components can be developed and tested independently, then integrated into the full question-answering pipeline.

The system uses Python for its core logic, transformer-based language models for selected agent tasks, sentence embeddings for vector retrieval, NetworkX for graph storage and traversal, and evaluation utilities to measure performance. A configurable execution path also allows testing without loading every real model.

The implementation separates data preparation, graph construction, retrieval, routing, answer generation, and graph maintenance. This structure makes components easier to test and improve without redesigning the entire pipeline.

6.2 Programming Language and Development Environment

6.2.1 Python

Python is AGENT-KGRAPH's primary programming language. Its libraries support natural-language processing, graph operations, model inference, data processing, and evaluation, making it suitable for implementing the agents and connecting them through a shared pipeline.

Python modules handle configuration, preprocessing, graph storage, triple extraction, verification, entity resolution, retrieval, routing, answer generation, feedback, and evaluation. The package also supports exposing operations through a command-line interface.

6.2.2 Modular Project Structure

The project uses a package-based structure under the `agentkgraph` directory. The main modules and their responsibilities are summarized below.

Module	Responsibility
config.py	Centralized configuration of models, thresholds, and system parameters
data/preprocess.py	Text chunking, sentence splitting, deduplication, and passage identifiers

kg/graph_store.py	Graph storage, relationships, confidence, provenance, and graph maintenance
agents/extractor.py	Candidate triple extraction
agents/verifier.py	Verification of candidate triples
agents/merger.py	Entity resolution and graph merging
agents/feedback.py	Confidence assessment and evolution workflow
retrieval/vector.py	Dense vector retrieval
retrieval/kg_paths.py	Entity linking, graph traversal, and path scoring
routing/router.py	Selection among vector, graph, and hybrid retrieval
generation/synthesizer.py	Evidence-based answer generation
pipeline.py	Integration of the major processing components
evaluation/metrics.py	Evaluation metrics and analysis utilities
main.py	Command-line entry point for supported operations

This structure separates the responsibilities of individual modules while allowing them to work together through the main pipeline.

6.3 Libraries and Frameworks Used

The project coding reference lists the following dependencies:

- **PyTorch** provides tensor operations and model execution for transformer-based components.
- **Hugging Face Transformers** loads and runs compatible pretrained models and tokenizers. It supports extraction, routing, and answer synthesis where configured.
- **PEFT** provides parameter-efficient fine-tuning tools, including LoRA. The design includes an optional LoRA workflow for the extractor, but this does not mean every proposed fine-tuning experiment has been completed.
- **bitsandbytes** supports low-bit model loading and quantization, including the proposed INT4 synthesizer configuration.
- **Accelerate** helps configure and run inference on supported hardware.
- **Sentence-Transformers** generates dense text embeddings for semantic retrieval. The project uses a BGE-small model for vector retrieval.
- **NetworkX** provides the in-memory graph representation and supports relationship traversal, breadth-first search, path exploration, and graph maintenance.
- **NumPy** supports numerical operations used in scoring and evaluation.
- **scikit-learn** provides utilities for classification, data processing, and evaluation.
- **psutil** supports system-resource monitoring, including process and memory measurements.
- **datasets** provides tools for working with supported machine-learning datasets and processing workflows.

The coding reference lists these dependencies in the project requirements. Record exact installed versions from the final project environment before submission.

6.4 Language Models and Embedding Model

6.4.1 Qwen2.5-1.5B-Instruct

The coding design specifies Qwen2.5-1.5B for triple extraction. The model turns source passages into candidate relationships for the verification and merger components.
Its structured output is expected to include the head entity, relationship, tail entity, and any supported metadata. This format helps validate extracted information before it changes the graph.

The coding reference also describes optional LoRA adaptation. Report training completion and metrics only when actual training records support those claims.

6.4.2 NLI-based Verification Model

The verification component checks whether a source passage supports a candidate triple. The design uses Natural Language Inference (NLI) for entailment checks, with a prompt-based fallback described in the coding reference.

Verification sits between extraction and graph modification. Candidates that fail the configured criteria are rejected rather than committed automatically. This reduces the risk of adding unsupported relationships.

A verifier's confidence score does not guarantee factual correctness. Its value depends on the model, source evidence, and acceptance threshold.

6.4.3 Qwen2.5-0.5B-Instruct Router

The routing component uses a lightweight Qwen2.5-0.5B model with contextual scoring. It examines query characteristics and selects among vector, knowledge-graph, and hybrid retrieval.

The implementation includes feature extraction, action scoring, and UCB-style exploration and exploitation. When a feedback signal is configured, these operations can also update routing preferences.

An adaptive routing mechanism does not, by itself, show that the router outperforms fixed strategies. That requires controlled comparisons.

6.4.4 BAAI/bge-small-en-v1.5

The project uses BAAI/bge-small-en-v1.5 to encode queries and passages as vectors for similarity search.

The methodology specifies a top-\(k\) value of four. The retrieval module ranks passages and returns the highest-scoring candidates for answer synthesis. Similarity indicates relevance, not proof that a passage supports an answer.

6.4.5 Qwen2.5-3B-Instruct Synthesizer

The proposed primary synthesizer is Qwen2.5-3B-Instruct with INT4 quantization. It generates a natural-language answer from the question and retrieved evidence.

The methodology calls for evidence-constrained generation and provenance-aware output. The synthesizer can receive passages, graph paths, or both. Where configured, the output may include passage identifiers or graph-path references to support traceability. These references help with inspection, but answer validity still needs to be assessed against the evidence and evaluation metrics.

6.5 Data Preprocessing and Passage Management

The preprocessing module prepares source material for graph construction and retrieval. It divides text into manageable chunks, splits passages into sentences, removes duplicates, and assigns passage identifiers.

These identifiers link processed text to later operations. Extracted triples can point back to their source passages, and retrieved passages can be identified during answer generation and evaluation.

Deduplication prevents repeated content from being indexed multiple times. The resulting passages can be used for embedding generation and candidate triple extraction.
MetaQA preprocessing and graph import were completed during earlier development. Dataset integration will validate and formalize that work, check mappings and identifiers, and extend the pipeline to HotpotQA and WebQSP. Those integrations should not be described as complete until validation is finished.

6.6 Knowledge-Graph Storage and Management

6.6.1 NetworkX Graph Representation

The coding design uses NetworkX as its primary in-memory graph representation. Entities are stored as nodes and relationships as edges. Depending on the implementation, attributes can hold confidence, provenance, timestamps, and other metadata.
This representation supports explicit relationship traversal for graph-based question answering. Unlike a plain collection of passages, it makes connections between entities directly available to retrieval.

The methodology also mentions optional Neo4j Community integration. Treat it as an optional storage alternative unless the final implementation confirms it has been configured and used.

6.6.2 Confidence and Provenance

Confidence metadata records the confidence assigned to relationships by the configured extraction and verification workflow. Provenance identifies the source passage or other evidence associated with a relationship.

The graph retriever can use confidence when ranking paths, and developers can use provenance to inspect the supporting evidence.

6.6.3 Graph Decay and Pruning

The graph-storage component can reduce the confidence of selected edges and prune entries that fall below configured criteria. These operations are intended to support graph maintenance as the system evolves.

Decay and pruning need a defined policy. A low-confidence edge is not necessarily false, and an edge should not be removed solely because it is rarely retrieved. The thresholds and effects of these operations require testing.

6.7 Retrieval and Query Routing

6.7.1 Vector Retrieval

The vector-retrieval module encodes a query, compares it with indexed passage representations, and returns the highest-ranked passages. The methodology sets the top-\(k\) value to four.

This approach is useful when a question needs descriptive text or uses different wording from the source. The retrieved passages provide context for answer synthesis.

6.7.2 Knowledge-Graph Retrieval

The graph-retrieval module identifies entities in a query and matches them to graph nodes. It then uses breadth-first search to explore connected relationships up to a configured hop limit. The project focuses on two-hop paths. Candidate paths are scored using available confidence information for their entities and edges. A configured minimum path score can filter lower-scoring candidates.

This approach supports questions that require connecting facts. Its performance depends on entity matching, graph coverage, and the relevance of the selected paths.

6.7.3 Hybrid Retrieval

Hybrid retrieval combines relevant passages from vector search with paths from the knowledge graph. The methodology describes score normalization and a configurable ranking strategy for selecting the combined evidence.
The approach uses semantic retrieval and explicit graph relationships together. It can add processing cost and introduce irrelevant evidence if results are not ranked appropriately.

6.7.4 Routing Controller

The routing controller chooses among vector, knowledge-graph, and hybrid retrieval. It uses query features and a scoring mechanism to select an action.

The coding reference describes contextual features, model-based scores, and UCB-style action selection and updates. A dry-run mode allows pipeline testing without loading every real model.

The selected action determines which retrieval operation runs. Comparative experiments are needed to determine whether this approach improves accuracy, latency, or both over fixed retrieval strategies.

6.8 Triple Extraction, Verification, and Entity Merging

Graph construction has three main steps: extraction, verification, and merging.
During extraction, the model proposes candidate triples from source passages. Each triple represents a possible relationship between two entities and can reference its supporting passage. During verification, the system checks whether the evidence supports each candidate. The verifier applies the configured entailment criteria and rejects unsupported triples. The

methodology describes one possible retry: a rejected extraction may be reformulated once before final rejection.

During entity resolution, the merger checks whether entity mentions match existing graph entities. Name normalization, matching, and duplicate handling help maintain consistency. Accepted relationships are merged with their associated metadata.

This sequence separates proposing a fact from accepting it. It also provides a point to reject invalid candidates before they change the graph.

6.9 Answer Synthesis and Self-Evolution

6.9.1 Answer Synthesis

The synthesis module receives the query and retrieved evidence. It builds the model context, applies the configured instructions, and generates a response.
Where supported, passage identifiers and graph-path information are retained as provenance. This lets developers inspect the evidence associated with an answer and investigate errors.

6.9.2 Confidence-Gated Feedback

The feedback module supports self-evolution. The methodology describes checking consistency across two answer samples, applying a confidence threshold, and optionally using user feedback. When the configured conditions are met, the system may propose candidate triples for verification.
Candidates must still pass verification and entity resolution before merging. This prevents every generated answer from becoming a trusted source of graph information.

6.9.3 Graph Update and Maintenance

Accepted triples are merged into the graph with available confidence and provenance information. The maintenance mechanism can then apply configured decay and pruning policies.

This process is intended to let the graph evolve through controlled updates. Its benefits need to be measured by comparing graph and answer quality before and after evolution.

6.10 Pipeline Integration and Execution

The `pipeline.py` module connects the main operations. The system supports offline corpus preparation and graph construction, followed by online query processing and answer generation.

The coding reference describes a command-line entry point for preprocessing, extractor training, graph construction, router training, evaluation, and serving. Which operations are available depends on the current project configuration.
Dry-run mode uses deterministic stub components to exercise the overall pipeline without loading every large model. This helps identify interface and integration problems before more expensive inference runs.

Real-model execution is used for selected experiments and performance checks. Report dry-run and real-model results separately: dry runs check pipeline behaviour, while real-model runs also test model loading and inference.

6.11 Testing and Evaluation Technologies

Automated tests check component behaviour and pipeline integration. The latest recorded full test run passed **71 tests**. This confirms that the covered test cases passed, but does not show that every input or production scenario has been validated.

The evaluation design includes:

- **Exact Match (EM):** Checks whether a predicted answer matches the reference under the benchmark’s normalization rules.
- **Token F1:** Measures token overlap between a prediction and its reference.
- **Hits@1:** Checks whether the correct target appears in the first position under the defined procedure.
- **Precision@k:** Measures the proportion of relevant items among the top-\(k\) results when relevance labels are available.
- **Provenance completeness:** Checks whether expected evidence references are present.
- **Routing analysis:** Examines selected retrieval actions and their relationship to question categories.
- **Latency:** Measures the time required for the relevant operations.
- **Resource measurements:** Assess computational use under the recorded test conditions.

The full evaluation plan also includes per-hop results, ablation studies, statistical testing, and comparative analysis. Report these as complete only when the corresponding experiments and outputs are available.

6.12 Preliminary Real-Model Experiments

The project has run model smoke tests to check that selected real models can load and execute. The recorded measurements are summarized below.

Component	Model	Recorded load time	Recorded inference time
Embedding	BAAI/bge-small-en-v1.5	9,867 ms	39.179 ms
Router	Qwen/Qwen2.5-0.5B-Instruct	9,146 ms	3,048.509 ms
Verifier	cross-encoder/nli-deberta-v3-base	8,346 ms	941.225 ms
Extractor	Qwen/Qwen2.5-1.5B-Instruct	14,002.864 ms	18,432.068 ms
Synthesizer	Qwen/Qwen2.5-3B-Instruct	32,192.471 ms	24,487.632 ms

The figures above are measurements from smoke-test runs, not a controlled comparison of model quality. They depend on the hardware, quantization settings, input lengths, and measurement procedure. A smoke test confirms that a model loaded and ran on the tested input; it does not establish accuracy or performance on other tasks.

The verified benchmark uses the MetaQA test split: 102 questions, with 34 each in the one-hop, two-hop, and three-hop categories. It used Qwen/Qwen2.5-0.5B-Instruct as the synthesizer, with 4-bit quantization disabled. The full text corpus was not loaded, and all questions used KG retrieval. We recalculated the results from the 102 records in reports/b9_metaqa_100_qwen05b_rerun/b9_end_to_end_metaqa.json. The run achieved 74.51% Exact Match, 83.54% Token F1, and a mean query latency of 7,684.04 ms. Hop-wise results appear in Section 7.6.

These results provide an initial baseline for this configuration. They should not be generalized to other datasets or model settings. Broader experiments and consistent comparisons are needed to measure the contributions of the individual retrieval strategies and the self-evolution mechanism.

6.13 Summary

AGENT-KGRAPH is a modular Python system that uses transformer models, dense text embeddings, NetworkX graph storage, and dedicated components for preprocessing, extraction, verification, entity merging, retrieval, routing, answer synthesis, and graph evolution.

The modular design supports incremental development and automated testing. The latest recorded full test run passed 71 tests. Preliminary real-model smoke tests and a MetaQA benchmark have also been completed, providing an initial engineering and experimental baseline.

Remaining work includes validating the existing MetaQA integration, adding HotpotQA and WebQSP, conducting comparative evaluation, and running systematic real-model experiments. The final report should distinguish these planned activities from work already completed.

7. RESULT DISCUSSION AND ANALYSIS

7.1 Introduction

This section reports results from implementing and testing AGENT-KGRAPH. It covers software tests, real-model smoke tests, a preliminary MetaQA question-answering run, results by question-hop category, and issues observed during execution.

The evaluation serves two purposes: checking that the modules and integrated pipeline run as expected, and measuring answer quality and response time on a benchmark sample. Passing software tests shows that the tested behaviours worked in that run. Benchmark metrics describe performance on the sampled questions. Neither establishes that the complete approach is better than alternatives.

We also checked the repository's validation records. The file reports/b9/final_validation.json is marked dry_run, and its real_model_benchmark_status is not_run. Its before-and-after evolution scores and kg_changes value are test fixtures, not observed MetaQA results, so they are excluded here. Although the implementation verifies candidate facts before adding them to the graph, no real query-session update log or paired predictions for static and evolving graphs were available. KG growth and static-versus-evolving performance therefore remain unmeasured.

The available results are preliminary. The MetaQA experiment covered 102 questions and selected knowledge-graph retrieval for every one. It provides an initial feasibility baseline, not a comprehensive comparison of retrieval modes.

7.2 Software Testing Results

The latest recorded full test run completed with 71 tests passed in 92.73 seconds.

Test result	Observed value
Tests passed	71
Total recorded execution time	92.73 seconds
Recorded test failures	0

The passing tests show that the behaviours covered by the suite worked in that run. Unit tests check individual operations, while integration tests check how modules work together. Together, they provide a regression baseline for future development.

A passing suite does not guarantee correct handling of every query, dataset, or model response. The conclusions depend on test coverage, input selection, and the environment in which the tests ran.

Before submission, rerun the tests against the exact code version being submitted. Keep the final commit identifier and test output so the results can be reproduced.

7.3 Real-Model Smoke-Test Results

Real-model smoke tests were conducted to check whether selected models could be loaded and execute representative inference operations. The recorded measurements are summarized below.

Component	Model	Load time	Inference time
Embedding	BAAI/bge-small-en-v1.5	9,867 ms	39.179 ms
Router	Qwen/Qwen2.5-0.5B-Instruct	9,146 ms	3,048.509 ms
Verifier	cross-encoder/nli-deberta-v3-base	8,346 ms	941.225 ms
Triple extractor	Qwen/Qwen2.5-1.5B-Instruct	14,002.864 ms	18,432.068 ms
Synthesizer	Qwen/Qwen2.5-3B-Instruct	32,192.471 ms	24,487.632 ms

The smoke tests confirmed that the listed components could perform representative inference under the recorded conditions. The extractor returned two ExtractedTriple items, and the synthesizer completed a sample generation.

These timings are individual measurements, not averages from a standardized workload. Model load time includes initialization and preparation; inference time measures execution on the tested input. Input length, hardware, cache state, quantization, and runtime settings can all affect the results.

The timings are not model-quality scores and should not be compared directly across components, which perform different tasks. Embedding generation, entailment verification, triple extraction, and answer synthesis have different inputs and outputs.

7.4 MetaQA Benchmark Setup

A preliminary end-to-end benchmark was run on a sample of the MetaQA test split. The sample contained 102 questions, divided equally across three hop categories.

Experimental setting	Recorded configuration
Dataset	MetaQA test split
Total questions	102
1-hop questions	34
2-hop questions	34
3-hop questions	34
Synthesizer	Qwen/Qwen2.5-0.5B-Instruct
4-bit quantization	Disabled
Retrieval action selected	Knowledge graph for all 102 questions
Full corpus loaded	No; the existing MetaQA graph was loaded

The experiment provides an initial end-to-end baseline using the existing graph and a real synthesizer. It does not evaluate the full MetaQA dataset or compare vector, graph, and hybrid retrieval, since every sampled question was routed to knowledge-graph retrieval.

7.5 Overall Benchmark Results

The recorded overall results are shown below.

Metric	Observed result
Evaluated questions	102 (34 per hop; verified)
Exact Match	74.51% (76/102)
Token F1	83.54%
Mean latency	7,684.04 ms/query
Questions routed to KG retrieval	102/102 (100%), KG retrieval

Exact Match (EM) checks whether the normalized predicted answer set matches the normalized reference set. Of the 102 saved predictions, 76 matched exactly (74.51%).

Token F1 is calculated from the overlap between predicted and reference entities, using precision and recall. The mean across the 102 questions was 83.54%.

Mean query latency was 7,684.04 milliseconds, calculated from the 102 per-question timings. Each timer covers the full engine.answer call, including any fallback work. Model and graph loading happen before timing begins. The total time for the separate benchmark invocation includes checkpoint writes and cleanup, so it is not the per-question mean.

All 102 questions used knowledge-graph retrieval. The benchmark therefore describes the KG-routed pipeline on this sample; it does not measure adaptive routing's benefit or show whether hybrid retrieval would improve the results.

7.6 Results by Number of Hops

The recorded hop-wise results are presented below.

Question category	Questions	Exact Match	Token F1	Mean latency
1-hop	34	97.06% (33/34)	98.24%	165.66 ms
2-hop	34	67.65% (23/34)	84.23%	3,753.91 ms
3-hop	34	58.82% (20/34)	68.15%	19,132.57 ms

The 1-hop group contains 34 questions. Recalculation gives 33 exact matches (97.06%), Token F1 of 98.24%, and mean per-question latency of 165.66 ms.
The 2-hop group contains 34 questions. Recalculation gives 23 exact matches (67.65%), Token F1 of 84.23%, and mean per-question latency of 3,753.91 ms.
The 3-hop group contains 34 questions. Recalculation gives 20 exact matches (58.82%), Token F1 of 68.15%, and mean per-question latency of 19,132.57 ms. The three hop groups contain 33 + 23 + 20 = 76 exact matches, so overall EM is 76/102 = 74.51%. The overall latency mean is the equally weighted mean of the three hop means because each group has 34 questions: (165.659 + 3,753.908 + 19,132.567)/3 = 7,684.045 ms, reported from full precision as 7,684.04 ms. Latency is measured around the full engine.answer call for every question, including unusually long calls. Five questions took more than 100 seconds: one 2-hop question and four 3-hop questions. The benchmark-wide elapsed time is measured separately and includes checkpoint writes and cleanup after model and graph loading, so it does not equal the sum of the 102 query timings.

The previously reported 3-hop results (61.76% EM, 74.97% F1, and 3,045.09 ms) came from a different run, reports/b9_metaqa_100_qwen05b. This section reports only the rerun that matches the overall results.

7.7 Error Analysis

The benchmark and implementation review identified several recurring error patterns.

7.7.1 Extra Entities in Answers

Some answers included entities beyond those expected. For example, a question about Mimsy Farmer and co-writers produced both Barbet Schroeder and Ernest Thompson, though the expected answer was more limited.

Extra entities can lower Exact Match even when part of an answer is correct. Relation-sequence constraints and answer-type validation may help address this.

7.7.2 Incomplete Answer Lists

Some questions required several films, people, or other entities, but the response included only some of them. This may happen when relevant paths are missing, path selection excludes useful candidates, or answer extraction fails to combine results.

Improving path coverage and checking list completeness are possible approaches. Their effectiveness needs to be tested.

7.7.3 Relation Labels Returned as Answers

Some failures returned labels such as `Directed_by`, `Director`, or `written_by` instead of the expected entity or value. This suggests that the fallback answer extractor can select a relationship label when it should return a node or a value from a path.

Answer-type checks and safeguards for failed path extraction may reduce these errors.

7.7.4 Formatting and Normalization Differences

An answer may contain the expected entities but still fail Exact Match if its format differs from the reference or the evaluator does not normalize that difference. List order, aliases, punctuation, and extra text can all affect exact comparisons.

The evaluator should be checked against the benchmark's intended scoring procedure. Keep the original predictions so that overly permissive normalization does not hide genuine errors.

7.7.5 High-Latency Cases

Five query calls in the verified log exceeded 100 seconds:

- `metaqa_2hop_test_0000016`: 112.43 s

- `metaqa_3hop_test_0000009`: 152.78 s
- `metaqa_3hop_test_0000013`: 175.10 s
- `metaqa_3hop_test_0000023`: 142.83 s
- `metaqa_3hop_test_0000032`: 159.17 s

These timings cover each full `engine.answer` call. The log does not identify what caused the long calls, so their cause remains unknown. Because they affect the mean, latency should also be considered alongside the full per-question distribution.

The comparison workbook reports median and 95th-percentile latency based on question-level measurements. Further analysis should use the same definition for full `engine.answer` latency, while measuring model-loading time and per-method resource use separately.

7.8 Comparative Analysis: What the Current Results Establish

A valid comparison requires testing competing configurations on the same questions under consistent conditions. The evaluation addendum reports vector-only, KG-only, and hybrid runs using the same 102 MetaQA question IDs.The originally saved router run is also included; because all 102 decisions selected KG, its results equal the KG-only run and do not measure routing benefit.

The following table separates the comparisons supported by current evidence from those that remain pending.

Comparison or claim	Current evidence	Conclusion
Software component execution	71 tests passed in the recorded full run	Covered tests passed
Real-model execution	Five component smoke tests recorded	Representative model inference succeeded
End-to-end question answering	102-question MetaQA sample	Initial feasibility demonstrated
1-hop versus multi-hop performance	Recorded per-hop metrics	Lower Exact Match was observed for 2-hop and 3-hop questions than for 1-hop
Vector versus KG retrieval	No comparable results in this benchmark	No performance winner established
KG versus hybrid retrieval	No comparable results in this benchmark	No performance winner established
Adaptive router versus fixed routing	All 102 questions used KG retrieval	Routing benefit not established
Static graph versus evolved graph	No controlled before/after comparison recorded	Self-evolution benefit not established
AGENT-KGRAPH versus published systems	No matched experimental setup	Superiority cannot be claimed

The results show that the integrated system can execute a real-model question-answering workload and achieve measurable performance on the selected MetaQA sample. They also expose limitations in multi-hop answer generation and latency stability.

These results do not show that AGENT-KGRAPH outperforms vector-only systems, hybrid retrieval baselines, or published methods. Published studies often use different datasets, sample sizes, models, evaluation protocols, and hardware. Comparing their results directly with this sample would not be a controlled comparison.

7.9 Limitations and Threats to Validity

Several limitations define how these results should be interpreted:

- **Sample size:** The benchmark contains 102 questions, split evenly across one-hop, two-hop, and three-hop categories. It does not establish performance on the full MetaQA test set.

- **Routing:** In the original routed benchmark, the router selected KG retrieval for all 102 questions. The addendum reports separate fixed-route vector-only and hybrid comparisons, but the router run does not measure the benefit of routing.

- **Model configuration:** The benchmark used Qwen2.5-0.5B-Instruct for synthesis, with 4-bit quantization disabled. Results may differ with other models, quantization settings, or inference configurations.

- **Answer normalization:** Exact Match depends on the evaluator's normalization rules. Errors from extra entities or formatting differences should be distinguished from retrieval failures.

- **Latency outliers:** A few slow calls substantially increased mean latency. The comparison workbook also reports median and 95th-percentile latency. Timing covers the full `engine.answer` call but excludes model and graph loading. Peak RSS was measured across the shared run, not separately for each method.

- **Dataset coverage:** The reported benchmark does not include HotpotQA or WebQSP, so it cannot establish generalization across datasets.

- **Ablations:** Controlled ablation studies have not yet isolated the effects of routing, hybrid retrieval, verification, provenance, or self-evolution.

These limitations do not invalidate the observed results. They set boundaries on the conclusions and point to experiments needed to strengthen them.

7.10 Planned Comparative Experiments

The vector-only, KG-only, and hybrid comparison on 102 questions is complete, and question-level outputs have been retained. The original router selected KG for every question, so that run does not provide an independent comparison of routing strategies. The following work remains:

1. **Extend retrieval comparisons:** Evaluate additional question samples and report uncertainty.

2. **Evaluate routing:** Record each selected action and construct a routing confusion matrix if reference action labels can be defined consistently.

3. **Expand hop-wise evaluation:** Evaluate more questions and report results separately for one-hop, two-hop, and three-hop questions.

4. **Test self-evolution:** Compare static and evolved graphs under controlled conditions. Measure whether graph updates improve answer quality without adding too many incorrect facts.

5. **Run component ablations:** Disable or replace selected components to assess graph-path retrieval, verification, provenance, and confidence gating.

6. **Analyze latency and resource use:** Measure median and high-percentile latency, model-loading time, and resource consumption using a consistent procedure.

7. **Add datasets:** Validate the existing MetaQA preparation, then integrate HotpotQA and WebQSP before drawing broader conclusions about generalization.

8. **Apply statistical analysis:** Where paired results are available, use suitable methods such as bootstrap confidence intervals and McNemar's test for paired binary correctness outcomes.

Retain question-level predictions and configuration details for each experiment so that results can be reproduced and compared fairly.

Table I — Design-parameter matrix of related systems vs. AGENT-KGRAPH

Framing for Section 3.6 ("Positioning"): every column is a parameter the proposed system is compared on; "✓ / ✗ / ◐" indicates capability present/absent/partial as stated by the cited work.

System	KG construction	Agentic curation	Self-evolution / updates	Query routing / adaptivity	Explainability / provenance	Multi-hop support	Locality / scale	Benchmarks	Reported headline result
Vanilla RAG [22]	✗ (static passage index)	✗	✗	✗ (single retriever)	◐ (retrieved passages only)	✗	220M–400M BART, FAISS	Wikipedia ODQA	factual grounding; needs joint retriever–generator training to transfer domains [1]
RAG-end2end [1]	✗	✗	✗ (updated only during	✗	◐	✗	joint-trained retriever+generator	COVID-19, News, Conversations	significant gains over original

			training)						RAG via joint domain adaptation [1]
In-Context RALM [2]	✗	✗	✗	✗	✗	✗	LLaMA-13B zero-shot	NQ, TriviaQA	+18 pts NQ (12.0 →31.0), +>5 pts TriviaQA (54.8 →60.1) with 2 docs [2]
FIT-RAG [3]	✗	✗	✗	◐ (self-knowledge recognizer)	✗	✗	Llama2-13B-Chat, black-box	TriviaQA, NQ, PopQA	+14.3% / +19.9% / +27.5% vs. no-augmentation [3]
GraphRAG survey formalization [4]	◐ (formalism only)	✗	✗ — incremental updates an **open problem** [4]	✗	◐ (graph-guided)	✓ (multi-hop via graph structure)	n/a (survey)	n/a	formalizes Graph-Based Indexing, Graph-Guided Retrieval, Graph-Enhan

									ced Generation [4]
Document GraphRAG [5]	✓ document-structure KG + keyword semantic linking	✗	✗	✗	◐	✓ (multi-hop benefits most)	local, task-dependent tuning	SQuAD, HotpotQA, manufacturing	consistent gains over naive RAG; optimizations for chunk size / keyword density / top-k are task-dependent [5]
Training-free KG-RAG [6]	✗ (uses existing KG)	✗	✗	✗	✓ (explicit reasoning chains)	✓ (decomposition)	zero-training, ICL/CoT	MetaQA	increased multi-hop accuracy, slight single-hop trade-off [6]
KnowledgeNavigator [7]	✗	✗	✗	◐ (iterative path search)	✓ (optimal-path search)	✓	LLM + KG, swappable	3 benchmarks	outperforms all LLM-based baselines [7]
KGP [8]	✓ doc KG (semantic/lexical edges)	◐	✗	◐ (traversal policy)	◐	✓	LLM + graph	multi-document QA	graph reduces retrieval latenc

									y; traversal agent guards quality [8]
KG-EGV [9]	✗	✓ LLM as generator/re-ranker/evaluator/verifier	◐ (backward updating)	✗	✓ (evidence synthesis)	✓	LLM + KG	FactKG, MetaQA, NQ, WebQ, TriviaQA	SOTA answer accuracy and evidence quality [9]
LQR [10]	✗	✗	✗	✓ complexity classification + NLI relevance	◐	✓ (multi-hop levels)	smaller models aid LLM	HotpotQA + single/multi-hop sets	**+9.4% acc, +16.14% F1 vs. Adaptive-RAG** on HotpotQA [10]
Reflective-tag RAG [11]	✗	✗	✗	✓ parallel doc scoring + reflective tags	◐	◐	LLM	complex-task QA	reduces hallucination by filtering low-quality docs [11]
Agentic RAG [12]	✗	✓ goal reasoning, self-direction	◐ (self-directed)	✓ hierarchical multi-agent	◐	✓	multi-agent	healthcare/finance/education surveys	architectural survey; scalability flagged as

									challenge [12]
Hybrid multi-agent GraphRAG (e-gov) [13]	✓ LLM-generated structured graphs	✓ multi-agent	✗	◐ (module selection)	✓ explainable assistant	✓	multi-agent, web+local	e-government	trustworthy explainable QA in deployment [13]
KGR [14]	✗	✓ autonomous retrofitting agent	✗ (fixes outputs, not KG)	✗	✓	◐	Vicuna-13B	Simple Q, Mintaka, HotpotQA	Simple Q 46.0/46.9 EM/F1; Mintaka 26.5/34.0; HotpotQA 10.2/20.6 [14]
LLM-based KG updater [15]	◐ (updates existing KG)	◐	✓ automatic fact refreshment	✗	◐	✗	LLM + RAG	KG update tasks	updates outdated facts via LLM reasoning + retrieval [15]
AGENT-KGRAPH (ours)	**✓ LLM-assisted extraction, verification and entity merging**	**✓ Extract–verify–merge workflow**	**✓ Confidence-gated updates and graph maintenance**	**✓ Vector/KG/Hybrid routing implemented**	**✓ Evidence/path provenance in QA pipeline**	**✓ Two-hop graph retrieval and hybrid retrieval**	**Local execution; memory limit not verified**	**MetaQA test sample (102 questions; 34 per hop); verified from saved question-level predictions and timings.**	**EM 74.51% (76/102); Token F1 83.54%; mean query latency**

									7,684.04 ms. Hop 3: 58.82 % EM, 68.15 % F1, 19,132.57 ms mean latency.

Table II — Reported Quantitative Results from the Literature and AGENT-KGRAPH

Setting	Model / System	Metric	Result
MetaQA 1-hop (34 questions)	AGENT-KGRAPH, Qwen2.5-0.5B-Instruct synthesizer	Exact Match (EM)	97.06% (33/34)
MetaQA 1-hop (34 questions)	AGENT-KGRAPH	Token F1	98.24%
MetaQA 2-hop (34 questions)	AGENT-KGRAPH, Qwen2.5-0.5B-Instruct synthesizer	Exact Match (EM)	67.65% (23/34)
MetaQA 2-hop (34 questions)	AGENT-KGRAPH	Token F1	84.23%
MetaQA 3-hop (34 questions)	AGENT-KGRAPH, Qwen2.5-0.5B-Instruct synthesizer	Exact Match (EM)	58.82% (20/34)
MetaQA 3-hop (34 questions)	AGENT-KGRAPH	Token F1	68.15%
MetaQA sampled test (102 questions)	AGENT-KGRAPH	Overall Exact Match (EM)	74.51% (76/102)
MetaQA sampled test (102 questions)	AGENT-KGRAPH	Overall Token F1	**83.54%**
MetaQA sampled test (102 questions))	AGENT-KGRAPH	Mean query latency	7,684.04 ms/query
MetaQA sampled test (102 questions)	AGENT-KGRAPH	Routing distribution	**102/102 queries; KG retrieval**
Automated test suite	AGENT-KGRAPH	Tests passed	**71 passed**
Retrieval-efficiency study	Stuff vs. Reciprocal RAG	Similarity / tokens / runtime	Reciprocal up to **91%** similarity; Stuff −7.2%

			score, **−71.3% tokens**, **+33.89% runtime**
Arxiv/BBC/NarrativeQA	LeanContext	Cost / ROUGE-1	cost −37.29%…−67.81%; ROUGE-1 −1.41%…−2.65%; +13.22%…+24.61% with open-source summarizers
MuSiQue composition analysis	multi-hop model sanity check	single-hop F1 drop / gap	**−30 F1 pts** under honest multi-hop settings; ≈**3× human–machine gap**

Note: Published results are reproduced from the cited studies and are provided as literature context; they are not results obtained by AGENT-KGRAPH. AGENT-KGRAPH's own results are reported separately using the project's sampled MetaQA evaluation. Controlled baseline comparisons, McNemar tests, and bootstrap confidence intervals will be included only after the corresponding experiments have been completed.

Table III — Component-Level Parameter Allocation

Component	Base model	Parameters	Quantization	Training signal	Output contract	Runtime role
Extractor agent	Qwen2.5-1.5B [27]	1.5B	Not confirmed; no quantization claim	Prompt-based triple extraction; fine-tuning not confirmed	Candidate subject–predicate–object triples	Offline KG construction and candidate triple generation
Verifier agent	cross-encoder/nli-deberta-v3-base, with configured prompt fallback	Model-specific; record exact count only if verified	none	Inference-based verification; no fine-tuning claim	Verification score / accept-reject decision	Triple verification during construction and evolution
Merger agent	canonicalization + fuzzy + LLM matching	Not applicable to rule-based components	Not applicable	Entity matching and merging logic	Canonical entities and merged graph facts	Entity resolution and duplicate handling
Router	Qwen2.5-0.5B classifier + UCB bandit [27]	0.5B	Not confirmed	Context features, routing scores and UCB updates;	Vector, KG or hybrid route with scores	Per-query retrieval-mode selection

				5K training labels not confirmed		
Dense retriever	BAAI/bge-small-en-v1.5	Approximately 33M; 384-dimensional embeddings	Not confirmed	Embedding inference; no fine-tuning claim	Ranked semantic passages	Vector and hybrid retrieval
Path retriever	NetworkX 2-hop BFS [30]	graph in RAM	Not applicable	Two-hop traversal, edge/path scoring and confidence thresholds	Scored graph paths and supporting evidence	KG and hybrid retrieval
Synthesizer	Qwen2.5-0.5B-Instruct for the recorded MetaQA benchmark; Qwen2.5-3B-Instruct also smoke-tested	0.5B for the benchmark run; 3B in a separate smoke test	4-bit quantization disabled in the recorded MetaQA rerun	Prompt-based answer synthesis; fine-tuning not confirmed	Generated answer from retrieved context	Final answer generation
Total	Multiple models and algorithmic components	**Not meaningfully additive across all components**	Configuration-dependent	Mixed inference, retrieval and graph-processing pipeline	**Integrated KG/vector/hybrid QA output**	Local execution pipeline; peak RAM not yet verified

Table IV — Baselines and Ablations Study Configuration

Variant	Construction	Key modification	Isolates / targets	Hypothesis served
Vanilla vector RAG [1]	Static passage index	Dense retrieval only; no KG traversal	Contribution of graph evidence	Planned
Path-only KG traversal	Static KG	KG retrieval only; no dense retrieval	Contribution of passage retrieval	Partially supported by the existing run; controlled comparison pending
Always-hybrid, no router	Static KG and vector index	Combine both retrieval modes using fixed routing	Contribution of adaptive routing	Planned
Training-free KG-RAG (reproduced)	Existing MetaQA KG	Reproduce the baseline's decomposition and	Head-to-head comparison on a	Planned; reproducible implementation required

		reasoning procedure	shared benchmark	
Document-GraphRAG-style [4]	Document-structure graph	Document-level and keyword-semantic linking	Effect of document-graph construction	Planned
Static-KG AGENT-KGRAPH	**KG built using the same pipeline**	Disable self-evolution after initial graph construction	Contribution of self-evolution	Planned
No-verifier curation	Same extraction and merging pipeline	Disable verifier-based accept/reject gate	Contribution of triple verification	Planned
No-decay	Same KG and update pipeline	Disable decay/pruning	Effect of graph maintenance	Planned
Router off	Same retrieval components	Replace adaptive routing with fixed hybrid retrieval	Contribution of adaptive routing	Planned

Note: The variants listed above define the planned baseline and ablation protocol. Except where explicitly supported by recorded experiment outputs, comparative effects have not yet been measured. All comparisons should use the same evaluation questions and metrics to isolate each component's contribution.

Table V — Evaluation Metrics and Results

Metric	Definition / computation	Evaluation status	Result / notes
Exact Match (EM)	Fraction of questions whose normalized predicted answer set equals the normalized reference answer set.	Measured	74.51% (76/102), MetaQA test sample
Token F1	For each question, calculate precision/recall F1 over the normalized predicted and reference answer sets using set overlap; report the mean across questions.	Measured	83.54% mean set Token F1 on the same 102 questions
EM / F1 per hop (1/2/3) [17]	Calculate EM and Token F1 separately for each hop category	Measured	1-hop: EM 97.06% (33/34), F1 98.24%; 2-hop: EM 67.65% (23/34), F1 84.23%; 3-hop: EM 58.82% (20/34), F1 68.15%.
Hits@1	Fraction of questions where the correct answer is ranked first	Not yet confirmed	Requires a defined ranking/evaluation procedure
Retrieval Precision@k [17]	Relevant retrieved evidence among the top-k results, based on labelled relevance	Planned	Requires evidence annotations and a fixed k

Path validity	Automated check that every retrieved KG path edge exists in the source graph in the correct direction; report the audited count and rate.	Automated full hybrid-path check measured; human spot-check pending	Hybrid: 5,062/5,062 paths and 14,562/14,562 edge steps valid (100%); see addendum
Provenance completeness [19]	Report citation-ID validity (IDs that resolve to retrieved evidence) and answer coverage (answers with at least one valid citation) separately.	Partly measured	Hybrid: 3,412/3,412 citation IDs valid (100%); 97/102 answers covered (95.10%). Vector: 0/102 cited. KG-only/proposed evidence not retained.
Answer faithfulness [19]	Split each generated answer into atomic factual claims. Two independent raters judge each claim against retrieved passages and KG paths as Supported, Partially supported, Unsupported, or Contradicted; adjudicate disagreements. Score Supported claims / all labeled claims.	Pending human claim-level review	No faithfulness score until annotations are complete; KG additions, merges, rejections, and pruning are reported separately under KG growth.
KG growth	Change in KG node and edge counts, with accepted, rejected, merged, and pruned triples recorded separately per query/session against a timestamped baseline graph.	Pending measurement	Static and evolution-test snapshots each contain 43,234 nodes and 133,582 edges; files are identical. Accepted/rejected update counts and session growth are unavailable.
Routing accuracy [19]	Agreement between router predictions and gold routing labels; confusion matrix	Planned	The current run routed all 102 queries to KG, which is insufficient to establish routing accuracy
Latency	Arithmetic mean of the 102 per-question wall times; timer surrounds the full engine.answer call, including work performed inside that call. Model/graph loading is outside the query timer.	Measured	7,684.04 ms/query overall; 1-hop 165.66 ms, 2-hop 3,753.91 ms, 3-hop 19,132.57 ms
Peak RAM	Maximum process memory during evaluation	Measured for the shared comparison process; not isolated by method	Peak process RSS: 4.25 GB across the combined comparison run.
Statistical significance	Paired tests and confidence intervals across controlled comparisons	Planned	Requires comparable predictions and repeated or paired evaluations

Automated test suite	Number of software tests passed	Measured	71 passed; this is a software validation result, not a QA accuracy metric

***Note:** A metric is labeled measured only when project results are available; otherwise, it remains a planned evaluation criterion. The MetaQA results come from a preliminary 102-question sample, not a controlled comparison with published baselines.*

Table VI — Gap-Coverage Matrix of Existing Systems and AGENT-KGRAPH

Required capability	Survey GraphRAG [1]	KG-RAG [6]	Doc-GraphRAG9	LQR [9]	KGR [14]	KG updater [1] [6]	**AGENT-KGRAPH**
KG built by LLM agents	✗	✗	◐	✗	✗	◐	✓
Verifier-gated commit-or-reject	✗	✗	✗	◐	✗	◐	✓
KG evolves with queries/feedback	✗	✗	✗	✗	✗ (updates outputs, not KG)	◐ Batch updates	✓ **Mechanism implemented; impact not yet measured**
Adaptive routing (learned)	✗	✗	✗	✓	✗	✗	◐ **Router implemented; comparative evaluation pending**
Provenance-cited answers	✗	✓ Reasoning chains	◐	✗	✓	✗	✓ **Evidence/path provenance implemented**
Fully local (<6 GB, no APIs)	n/a	◐	✓	◐	◐ (13B)	◐	◐ **Local execution; complete API and peak-memory claims need verification**
Incremental-update open problem addressed	listed as open [1]	✗	✗	✗	✗	◐	✓ **Update pipeline implemented; effectiveness not yet established**

MetaQA Evaluation Addendum

This addendum documents the completed retrieval comparison on 102 questions and the limits of the results. Each retrieval mode used the same question IDs, with 34 questions in each hop category. We calculated Exact Match and Token F1 from question-level predictions and references, and recomputed mean latency from per-question timing records.

The full vector-only and hybrid outputs are in MetaQA Retrieval Comparison Raw Results.json. The original KG-only/proposed-router run is in MetaQA KG-Routed Baseline Raw Results.json. The workbook contains summary tables, figures, and human-review sheets.

The vector index contains 134,741 MetaQA knowledge-graph triples, converted into short fact passages. It is a vector search over KG facts, not narrative documents. Vector-only and hybrid modes used fixed route overrides and skipped router inference; the saved KG-only/proposed run includes the router.

Answer synthesis followed the existing engine behavior: it used KG-path answer extraction when paths were available and otherwise used the Qwen2.5-0.5B-Instruct fallback. Peak process RSS was 4.25 GB across the shared run, so it is not an isolated measurement for any one method.

Table VII. Triple Extraction Precision, Recall, and F1

Evaluation item	Observed result	Status / interpretation
HotpotQA source sample	30 source passages; reservoir sample, seed 2026	Prepared for manual review; not an extraction benchmark score
Extractor output	60 candidate triples; 2 source sentences yielded no candidate	Qwen2.5-0.5B-Instruct, temperature 0, max 128 new tokens; configured 1.5B extractor could not be loaded within available memory
Precision / Recall / F1	Pending human gold labeling and adjudication	Do not interpret as measured until candidate and omitted triples are reviewed

The accompanying workbook contains source sentences, predicted triples, and fields for human gold triples and adjudicated TP/FP/FN labels. Precision, recall, and F1 must be computed only after this review is complete.

Table VIII. Answer Performance on the shared 102-Question MetaQA Sample

Method	N	Exact Match (%)	Token F1 (%)	Mean latency (ms)
Vector-only	102	1.96	7.56	15,680.44
KG-only	102	74.51	83.54	7,684.04
Hybrid	102	74.51	83.54	8,297.68
Proposed router	102	74.51	83.54	7,684.04

The proposed-router result is the same saved run as KG-only because all 102 questions were routed to KG. It is not an independent method result. These are sample results, not full MetaQA test-set results, and no superiority claim is made.

Table IX. Hop-wise Exact Match and Token F1

Hop	Questions	Vector EM / F1 (%)	KG EM / F1 (%)	Hybrid EM / F1 (%)	Router EM / F1 (%)
1-hop	34	2.94 / 9.64	97.06 / 98.24	97.06 / 98.24	97.06 / 98.24
2-hop	34	0.00 / 1.18	67.65 / 84.23	67.65 / 84.23	67.65 / 84.23
3-hop	34	2.94 / 11.86	58.82 / 68.15	58.82 / 68.15	58.82 / 68.15

The measured KG/router hop EM counts are 33/34, 23/34, and 20/34, totaling 76/102 (74.51%). The three equally sized hop groups yield an overall mean latency of 7,684.04 ms for the saved KG/router run. The hop means are 165.66 ms, 3,753.91 ms, and 19,132.57 ms. All use the same full engine.answer timer and include slow calls.

Table X. Knowledge-Graph Snapshot and Evolution Evidence

Graph snapshot	Nodes	Edges	Accepted / rejected triples	Interpretation
Static MetaQA graph	43,234	133,582	Not logged	Reference snapshot
Evolution-test graph file	43,234	133,582	Not logged	File is byte-identical to static snapshot; no measured query-session growth is demonstrated

Graph 3 cannot be produced without timestamped node/edge counts after each processed query or session and corresponding update decisions. The equal snapshot files do not establish that an evolving graph was exercised.

Table XI. Evidence Quality

Metric	Automated result	What it establishes / remaining work
KG path validity (Hybrid)	5,062/5,062 paths; 14,562/14,562 edge steps valid (100%)	Every returned path edge exists in the source graph in the correct direction; human spot-check sheet also provided
Citation-ID validity (Hybrid)	3,412/3,412 IDs matched retrieved evidence (100%)	Identifier consistency, not proof that each citation semantically supports its claim
Answer citation coverage (Hybrid)	97/102 answers (95.10%) had at least one valid citation	Coverage only; does not establish claim-level faithfulness
Vector citation coverage	0/102 answers (0%) had citations	No provenance retained in those vector outputs

Metric	Automated result	What it establishes / remaining work
Answer faithfulness	Pending human claim-level review	Define atomic factual claims; two independent raters label each Supported, Partially supported, Unsupported, or Contradicted against retrieved evidence; adjudicate disagreements; score Supported / all labeled claims
KG changes	Report separately in Table 4	Do not use added/merged/rejected/pruned triples as an answer-faithfulness measure

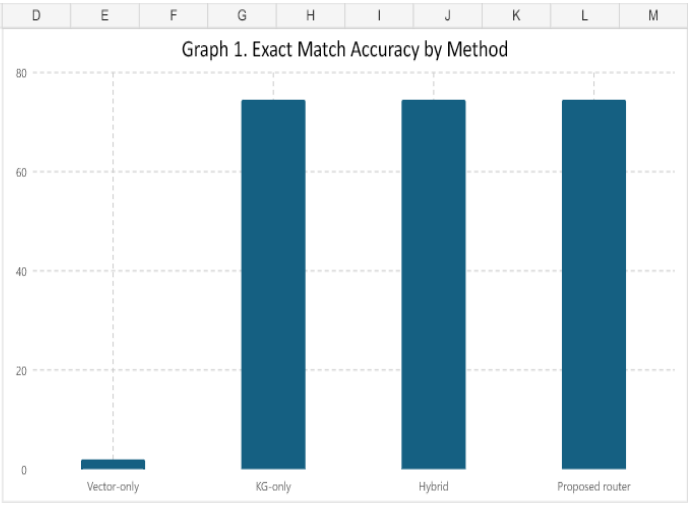
Table XII. Efficiency Results

Method	Mean response time (ms)	Accuracy (EM, %)	Peak RAM (GB)
Vector-only	15,680.44	1.96	4.25*
KG-only	7,684.04	74.51	4.25*
Hybrid	8,297.68	74.51	4.25*
Proposed router	7,684.04	74.51	4.25*

*Peak RAM is the peak RSS of the shared full comparison process, not separately measured per method. Fixed vector/hybrid modes bypass router inference while the saved KG/proposed run includes it; latency comparisons should be read with that timing difference in mind.

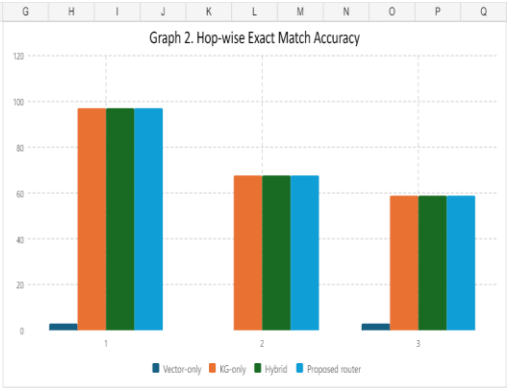
Graph 1. Accuracy Comparison on the shared MetaQA Sample

Method	EM (%)
Vector-only	1.96
KG-only	74.51
Hybrid	74.51
Proposed router	74.51



Graph 2. Hop-Wise Exact Match Comparison

Hop	Vector-only	KG-only	Hybrid	Proposed router
1	2.94	97.06	97.06	97.06
2	0.00	67.65	67.65	67.65
3	2.94	58.82	58.82	58.82



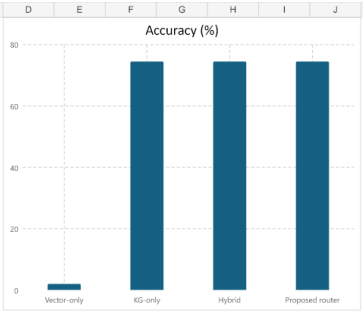
Graph 3 and 4 Status

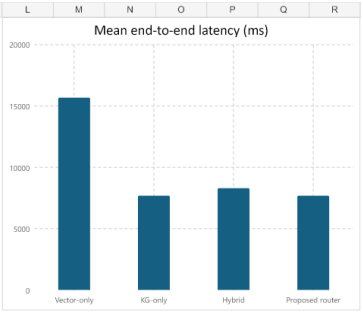
Requested graph	Status	Source data still required
Graph 3. KG nodes/edges vs. processed queries	Not produced; no time-series values available	Timestamped graph node/edge counts and accepted/rejected update logs by query or session
Graph 4. Static vs. evolving answer accuracy by session	Not produced; no paired session results available	Per-session predictions for static and evolving KG on identical question IDs, with matching model/settings

Graph 5. Accuracy and Mean Latency by Method

Method	Accuracy (%)
Vector-only	1.96
KG-only	74.51
Hybrid	74.51
Proposed router	74.51

Method	Mean latency (ms)
Vector-only	15680.44
KG-only	7684.04
Hybrid	8297.68
Proposed router	7684.04





Experimental conditions and limitations: All four reported modes use the same 102 question IDs. Triple-extraction and answer-faithfulness scores are not reported because they have not been established through human review. A static-versus-evolving accuracy comparison is also unavailable. The annotation workbook uses a fixed sampling seed of 2026 to select items for review; this is not a three-seed model experiment. Calculating means and standard deviations across three random seeds remains future work. The two raw JSON files retain question-level predictions and timings for the compared modes. The workbook contains sampled evidence, review fields, and formulas for continuing the human evaluation.

7.11 Summary

The implementation and evaluation show that AGENT-KGRAPH can run an end-to-end question-answering workflow with real language models and a knowledge graph. The system combines preprocessing, graph construction, triple extraction and verification, entity resolution, vector and graph retrieval, query routing, answer synthesis, provenance, and controlled graph evolution.

The first five roadmap batches are complete. They established the project foundation, graph construction, retrieval, agentic question answering and routing, and self-evolution. Existing MetaQA preprocessing and graph import provide a starting point for the next dataset-integration stage.

The recorded software test run passed **71 tests in 92.73 seconds**. Real-model smoke tests also confirmed that the configured embedding, routing, verification, triple-extraction, and synthesis components could load and perform representative inference tasks. These results validate the behaviours covered by the tests, but do not by themselves establish the system's accuracy or overall effectiveness.

The verified end-to-end MetaQA benchmark contains 102 test questions, with 34 in each hop category, and uses Qwen/Qwen2.5-0.5B-Instruct for synthesis. Recalculation from the saved question-level records gives 74.51% Exact Match (76/102), 83.54% Token F1, and a mean per-question latency of 7,684.04 milliseconds. All 102 questions used knowledge-graph retrieval. By hop count, Exact Match was 97.06%, 67.65%, and 58.82%; Token F1 was 98.24%, 84.23%, and 68.15%; and mean latency was 165.66 ms, 3,753.91 ms, and 19,132.57 ms, respectively.

These results provide an initial feasibility baseline for the KG-based pipeline, not a comparison of all retrieval strategies.

Exact Match declined as the number of hops increased, from 97.06% for one-hop questions to 67.65% for two-hop and 58.82% for three-hop questions. Three-hop Token F1 was 68.15%. Error inspection found incomplete answer lists, extra entities, relationship labels returned instead of expected answer values, and formatting differences. The latency log also contains five individual query calls that took more than 100 seconds. These issues point to areas for further work: entity matching, path selection, answer extraction, output validation, and latency handling.

The results need to be interpreted in light of the study's limits. The benchmark covers a relatively small sample, and the router selected only KG retrieval. Broader dataset integration and controlled comparisons are still pending. The results therefore do not show that AGENT-KGRAPH outperforms vector-only or hybrid retrieval, or published systems. Nor has the contribution of self-evolution been established through a controlled comparison of static and evolving graphs.

The next stage will validate and formalize the existing MetaQA preparation, integrate HotpotQA and WebQSP, and expand evaluation under consistent conditions. Planned work includes comparing vector, KG, hybrid, and router-selected retrieval; analyzing performance by hop count; running component ablations; assessing provenance completeness; and measuring latency and resource use more systematically. Statistical analysis should follow when enough paired results are available.

These results establish a working technical foundation for AGENT-KGRAPH and provide measurable evidence of its current behaviour. They also identify limitations that can guide further development. The project is now an implemented and tested research prototype. Broader dataset validation, comparative evaluation, and evidence for the benefits of adaptive routing and self-evolution remain future objectives.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

AGENT-KGRAPH, titled *Self-Evolving Agentic Knowledge-Graph Curation with Hybrid KG–Vector Retrieval*, is a modular question-answering system that combines structured knowledge, semantic retrieval, agentic decision-making, answer synthesis, and controlled graph updates. It addresses the limits of relying on one retrieval method by combining knowledge-graph and vector retrieval, with a router that selects graph, vector, or hybrid search.

The project has completed the first five batches of its roadmap: establishing the foundation, building the graph-construction and retrieval pipelines, integrating question answering and routing, and implementing self-evolution. The architecture includes triple extraction and verification, entity resolution and merging, graph-path retrieval, evidence-based answer synthesis, provenance handling, and graph maintenance. Its modules can be developed and tested independently, then connected through the integrated pipeline.

The recorded full test run passed 71 tests in 92.73 seconds. Real-model smoke tests also confirmed that selected embedding, routing, verification, extraction, and synthesis components could perform representative inference tasks.

We verified the 102-question MetaQA benchmark using its saved predictions, references, and latency records. It achieved 74.51% Exact Match (76/102), 83.54% Token F1, and a mean per-question latency of 7,684.04 ms. By hop count, Exact Match was 97.06%, 67.65%, and 58.82%; Token F1 was 98.24%, 84.23%, and 68.15%. Every question in this run used knowledge-graph retrieval. These results provide a KG-pipeline baseline, not a comparison of all retrieval strategies.

The results also point to areas for improvement. Longer relationship chains were more challenging than one-hop questions. Error analysis found incomplete answer lists, extra entities in generated answers, relationship labels returned instead of expected answers, and unusually slow fallback cases. These issues point to further work on entity matching, graph-path selection, answer extraction, output validation, and latency.

AGENT-KGRAPH has progressed from a design to an implemented and tested research prototype. The results show that its main components can run together in a question-answering workflow and provide a baseline for further experiments. They do not establish that the architecture outperforms alternative retrieval approaches; controlled comparisons and broader evaluation are still needed.

8.2 Future scope

The roadmap includes the following development and evaluation work:

1. **Dataset integration and validation:** Formalize and validate the existing MetaQA preprocessing and graph import. Add HotpotQA and WebQSP, including dataset-specific preprocessing, question-answer mappings, passage identifiers, and validation through the integrated pipeline.

2. **Comparative evaluation:** Evaluate vector-only, graph-only, hybrid, and router-selected retrieval on the same questions under consistent conditions. Compare Exact Match, Token F1, applicable retrieval metrics, provenance completeness, and latency to measure the contribution of each strategy.

3. **Multi-hop reasoning:** Improve entity linking, graph-path ranking, candidate filtering, and answer extraction for questions involving multiple relationships. Validate answer types and list completeness to reduce cases where relationship labels or unrelated entities appear in responses.

4. **Routing and answer quality:** Compare contextual routing with fixed retrieval strategies to determine whether it provides a measurable benefit. Routing confusion matrices and question-category analysis can help show when each action is appropriate.

5. **Self-evolution:** Compare static and evolving knowledge graphs under comparable conditions. Measure how confidence gating, triple verification, entity merging, decay, and pruning affect graph consistency and answer quality.

6. **Performance and resource use:** Investigate high-latency fallback cases. Measure median and high-percentile latency, model-loading overhead, and resource consumption under defined conditions. Assess how model configurations and inference settings affect response time and answer quality.

7. **Final integration and demonstration:** Complete the planned evaluation, consolidate documentation, review the implementation, prepare a demonstration, and organize the final submission. Update the conclusions as new results become available.

Further dataset integration and evaluation will help measure the architecture and its components, and clarify its performance and practical limitations.

9. REFERENCES

[1] B. Peng *et al*., "Graph Retrieval-Augmented Generation: A Survey," *ACM Transactions on Information Systems*, vol. 44, no. 2, pp. 1–52, 2025, doi: 10.1145/3777378.

[2] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024, doi: 10.1109/TKDE.2024.3352100.

[3] J. Linders and J. M. Tomczak, "Knowledge graph-extended retrieval augmented generation for question answering," *Applied Intelligence*, vol. 55, no. 17, 2025, doi: 10.1007/s10489-025-06885-5.

[4] S. Knollmeyer, O. Caymazer, and D. Großmann, "Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain," *Electronics*, vol. 14, no. 11, Art. no. 2102, 2025, doi: 10.3390/electronics14112102.

[5] Y. Wan, Z. Chen, Y. Liu, C. Chen, and M. Packianather, "Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing," *Advanced Engineering Informatics*, vol. 65, Art. no. 103212, 2025, doi: 10.1016/j.aei.2025.103212.

[6] L. Mariotti, V. Guidetti, F. Mandreoli, A. Belli, and P. Lombardi, "Combining large language models with enterprise knowledge graphs: a perspective on enhanced natural language understanding," *Frontiers in Artificial Intelligence*, vol. 7, Art. no. 1460065, 2024, doi: 10.3389/frai.2024.1460065.

[7] Y. Wang *et al*., "Large Model-Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1906–1949, 2025, doi: 10.1109/COMST.2025.3576176.

[8] S. Liu and Y. Fang, "Use Large Language Models for Named Entity Disambiguation in Academic Knowledge Graphs," in *Proc. Atlantis Highlights in Computer Sciences*, 2023, pp. 681–691, doi: 10.2991/978-94-6463-264-4_79.

[9] J. Huang *et al*., "Layered Query Retrieval: An Adaptive Framework for Retrieval-Augmented Generation in Complex Question Answering for Large Language Models," *Applied Sciences*, vol. 14, no. 23, Art. no. 11014, 2024, doi: 10.3390/app142311014.

[10] Z. Wang, X. Liu, S. Li, and J. C. S. Lui, "Efficient Explorative Key-Term Selection Strategies for Conversational Contextual Bandits," in *Proc. AAAI Conf. Artificial Intelligence*, 2023, pp. 10288–10295, doi: 10.1609/aaai.v37i8.26225.

[11] W. Jin, B. Zhao, H. Yu, X. Tao, R. Yin, and G. Liu, "Improving embedded knowledge graph multi-hop question answering by introducing relational chain reasoning," *Data Mining and Knowledge Discovery*, vol. 37, no. 1, pp. 255–288, 2023, doi: 10.1007/s10618-022-00891-8.

[12] C. Peng, F. Xia, M. Naseriparsa, and F. Osborne, "Knowledge Graphs: Opportunities and Challenges," *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13071–13102, 2023, doi: 10.1007/s10462-023-10465-9.

[13] T. Guo *et al*., "KnowledgeNavigator: leveraging large language models for enhanced reasoning over knowledge graph," *Complex & Intelligent Systems*, vol. 10, no. 5, pp. 7063–7076, 2024, doi: 10.1007/s40747-024-01527-8.

[14] X. Guan *et al*., "Mitigating Large Language Model Hallucinations via Autonomous Knowledge Graph-Based Retrofitting," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 18126–18134, doi: 10.1609/aaai.v38i16.29770.

[15] D. Yu, C. Zhu, Y. Yang, and M. Zeng, "JAKET: Joint Pre-training of Knowledge Graph and Language Understanding," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 11630–11638, doi: 10.1609/aaai.v36i10.21417.

[16] T. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop Questions via Single-hop Question Composition,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022, doi: 10.1162/tacl_a_00475.
[17] S. Li *et al*., “HopRetriever: Retrieve Hops over Wikipedia to Answer Complex Questions,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 15, 2021, pp. 13279–13287, doi: 10.1609/aaai.v35i15.17568.
[18] T. Alkhaldi, C. Chu, and S. Kurohashi, “Flexibly Focusing on Supporting Facts, Using Bridge Links, and Jointly Training Specialized Modules for Multi-Hop Question Answering,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3216–3225, 2021, doi: 10.1109/TASLP.2021.3120643.
[19] V. Adlakha, P. BehnamGhader, X. H. Lù, N. Meade, and S. Reddy, “Evaluating Correctness and Faithfulness of Instruction-Following Models for Question Answering,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 681–699, 2024, doi: 10.1162/tacl_a_00667.
[20] Z. Zou, J. X. Huang, Z. Ren, and E. Kanoulas, “Learning to Ask: Conversational Product Search via Representation Learning,” *ACM Transactions on Information Systems*, vol. 41, no. 2, 2023, doi: 10.1145/3555371.
[21] J. Wei *et al*., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.
[22] P. Lewis *et al*., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
[23] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proc. 37th Int. Conf. Machine Learning*, vol. 119, 2020, pp. 3929–3938.
[24] Z. Yang *et al*., “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” in *Proc. Conf. Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.
[25] W. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, “The Value of Semantic Parse Labeling for Knowledge Base Question Answering,” in *Proc. 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing*, vol. 2, 2015, pp. 201–206.
[26] A. Q. Jiang *et al*., “Mistral 7B,” arXiv:2310.06825, 2023.
[27] Qwen Team, “Qwen2.5: A Party of Foundation Models,” Qwen technical report, 2024. [Online]. Available: https://qwenlm.github.io/
[28] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. Conf. Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.
[29] T. Wolf *et al*., “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
[30] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in *Proc. 7th Python in Science Conference*, 2008, pp. 11–15.

10. PLAG REPORT (Through Turnitin)
Similarity Plagiarism Report + AI Plagiarism Report